

4.1 引言 4.2 组合逻辑 4.3 结构化建模 4.4 时序逻辑 4.5 更多组合逻辑 4.6 有限状态机 4.7 数据类型* 4.8 参数化模块* 4.9 测试平台 4.10 总结 习题 面试问题

4.1 引言

到目前为止，我们一直专注于在原理图层面设计组合和时序数字电路。寻找一组高效逻辑门以执行特定功能的过程既费时又易出错，需要手动简化真值表或布尔方程，并手动将有限状态机（FSM）转换为门电路。在20世纪90年代，设计人员发现，如果他们在更高的抽象层次上工作，仅指定逻辑功能，并让*computer-aided design*（CAD）工具生成优化后的门电路，效率会高得多。这些规范通常以*hardware description language*（HDL）的形式给出。两种主流的硬件描述语言是*SystemVerilog*和*VHDL*。

SystemVerilog 和 *VHDL* 基于相似的原理，但语法不同。本章中对这两种语言的讨论分为两栏，以便进行字面并排比较，*SystemVerilog* 在左侧，*VHDL* 在右侧。首次阅读本章时，请专注于其中一种语言。一旦掌握了一种，如果需要，你很快就能掌握另一种。

后续章节将以原理图和HDL两种形式展示硬件。如果您选择跳过本章且不学习任何一种HDL，您仍能通过原理图掌握计算机组成原理。然而，当前绝大多数商业系统均采用HDL而非原理图构建。若您预期在职业生涯中从事数字设计工作，我们强烈建议您学习其中一种HDL。

4.1.1 模块

具有输入和输出的硬件模块称为*module*。与门、多路选择器和优先级电路都是硬件模块的示例。描述模块功能的两种通用风格是

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

有些人仍在使用Verilog，即SystemVerilog的原始形态。因此，我们在配套网站（见前言）上提供了描述Verilog的本章版本。

*behavioral*以及*structural*。行为模型描述模块的功能。结构模型描述模块如何由更简单的部分构建而成；这是层次化的一种应用。HDL示例4.1中的SystemVerilog和VHDL代码展示了计算示例2.6中布尔函数 $y = a \oplus b \oplus c$ 的模块的行为描述。在这两种语言中，该模块被命名为sillyfunction，具有三个输入a、b和c，以及一个输出y。

HDL示例4.1 组合逻辑

SystemVerilog	VHDL
<pre>模块 sillyfunction(input logic a, b, c, output logic y); 将 y = ~a & ~b & ~c a & ~b & ~c a & ~b & c; 结束模块</pre> 一个SystemVerilog模块以模块名称和输入输出列表开始。assignment语句描述组合逻辑。~表示非，&表示与， 表示或。 逻辑信号（如输入和输出）是布尔变量（0或1）。它们也可能具有浮空和未定义的值，如第4.2.8节所述。 SystemVerilog中引入了logic类型。它取代了reg类型，后者在Verilog中一直是混淆的根源。除了具有多个驱动器的信号外，logic应随处使用。具有多个驱动器的信号称为<i>nets</i>，将在第4.7节中解释。	<pre>库IEEE; 使用IEEE.STD_LOGIC_1164.all; 实体sillyfunction 是 端口(a, b, c: 输入 STD_LOGIC; y: 输出 STD_LOGIC); 结束; 架构synth of sillyfunction是开始 y <= (not a and not b and not c) or (a and not b and not c) or (a and not b and c); 结束;</pre> VHDL 代码包含三个部分：库使用子句、实体声明和结构体。库使用子句将在第 4.7.2 节中讨论。实体声明列出了模块名称及其输入和输出。结构体定义了模块的功能。 VHDL信号，如输入和输出，必须具有<i>type declaration</i>。数字信号应声明为STD_LOGIC类型。STD_LOGIC信号可以具有'0'或'1'的值，以及将在第4.2.8节中描述的浮动和未定义值。STD_LOGIC类型在IEEE.STD_LOGIC_1164库中定义，这就是必须使用该库的原因。 VHDL 在 AND 和 OR 之间缺乏良好的默认运算顺序，因此布尔表达式应使用括号。

模块，正如你所期望的那样，是模块化思想的一个良好应用。它具有定义明确的接口，包括其输入和输出，并执行特定的功能。只要模块能够实现其功能，其具体的编码方式对于可能使用该模块的其他部分来说并不重要。

4.1.2 语言起源

大学在首门课程教授这两种语言的比例几乎持平。行业正逐渐转向SystemVerilog，但许多公司仍在使用VHDL，因此许多设计人员需要熟练掌握这两种语言。

SystemVerilog Verilog 由 Gateway Design Automation 于 1984 年开发，作为一款专有逻辑仿真语言。1989 年 Gateway 被 Cadence 收购，1990 年 Verilog 成为开放标准，由 Open Verilog International 管理。该语言于 1995 年成为 IEEE 标准¹。2005 年，该语言进行了扩展，以简化特殊用法并更好地支持系统建模与验证。这些扩展已合并为一个单一语言标准，即现在的 SystemVerilog（IEEE STD 1800-2009）。SystemVerilog 文件名通常以 .sv 结尾。	VHDL VHDL是VHSIC Hardware Description Language的缩写。而VHSIC则是美国国防部Very High Speed Integrated Circuits项目的缩写。 VHDL最初由美国国防部于1981年开发，用于描述硬件的结构和功能。其根源可追溯至Ada编程语言。该语言最初设想用于文档编制，但很快被采纳用于仿真和综合。IEEE于1987年将其标准化，此后多次更新标准。本章基于VHDL标准的2008年修订版（IEEE STD 1076-2008），该版本以多种方式简化了语言。要在ModelSim中使用VHDL 2008，可能需要在modelsim.ini配置文件中将VHDL93 {v*}设置为2008。VHDL文件名通常以.vhd结尾。
---	---

与SystemVerilog相比，VHDL更为冗长且繁琐，正如你对委员会开发的语言所预期的那样。

两种语言都完全能够描述任何硬件系统，并且各有其独特之处。最佳选择是使用您所在站点已采用的语言，或客户要求的语言。如今大多数CAD工具允许混合使用这两种语言，以便不同模块可以用不同语言进行描述。

4.1.3 仿真与综合

HDL的两大主要用途是逻辑simulation和synthesis。在仿真过程中，将输入施加到模块，并检查输出以验证模块是否正确运行。在综合过程中，模块的文本描述被转换为逻辑门。

仿真

人类在日常工作中难免会犯错。硬件设计中的此类错误被称为bugs。从数字系统中消除这些缺陷显然至关重要，尤其是在客户支付费用且系统正确运行关乎生命安全的情况下。在实验室中测试系统非常耗时，而在实验室中发现错误原因可能极其困难，因为只有路由到芯片引脚的信号才能被观测到。

¹The Institute of Electrical and Electronics Engineers (IEEE) is a professional society responsible for many computing standards, including Wi-Fi (802.11), Ethernet (802.3), and floating-point numbers (754).

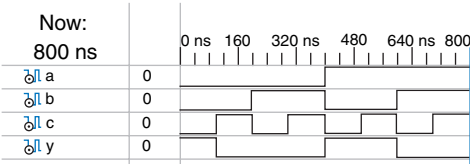
“bug”一词的出现早于计算机的发明。托马斯·爱迪生在1878年将他发明中的“小故障和困难”称为“bugs”。

第一个真正的计算机漏洞是一只飞蛾，它被夹在了 1947 年哈佛马克II型 机电计算机的继电器之间。 由格蕾丝·霍珀发现，她记录了这一事件，连同飞蛾本身以及注释“发现漏洞的第一个实际案例”。



Source: 笔记本条目，由美国海军历史中心提供；照片编号：NII 96566-KN

图4.1 仿真波形



无法直接观察芯片内部发生的情况。在系统构建完成后纠正错误，代价可能极其高昂。例如，修正尖端集成电路中的一个错误，成本超过一百万美元，且需要数月时间。英特尔奔腾处理器中臭名昭著的FDIV（浮点除法）漏洞，迫使公司在芯片已发货后召回，1994年总计损失达4.75亿美元。逻辑仿真对于在系统构建前进行测试至关重要。

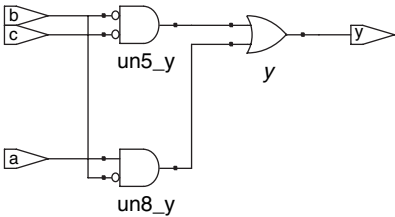
图4.1展示了先前sillyfunction模块的仿真²波形，表明该模块工作正常。根据布尔方程，当a、b、c为000、100或101时，y为真。

合成
逻辑综合将HDL代码转换为描述硬件的*netlist*（例如逻辑门及其连接线）。逻辑综合器可能会执行优化以减少所需的硬件量。网表可以是文本文件，也可以绘制成原理图以帮助可视化电路。图4.2展示了sillyfunction模块的综合结果。³请注意，三个三输入与门如何简化为两个两输入与门，正如我们在例2.6中使用布尔代数所发现的那样。

综合工具为每个综合出的门添加标签。在图4.2中，这些标签是un5_y、un8_y和y。

HDL中的电路描述类似于编程语言中的代码。然而，您必须记住，这些代码旨在表示硬件。SystemVerilog和VHDL是包含许多命令的丰富语言。

图4.2 综合电路



本章中的仿真使用ModelSim PE学生版10.3c进行。选择ModelSim是因为它在商业上被广泛使用，同时其容量为10,000行代码的学生版可免费获取。

本章中，综合均使用Synopsys公司的Synplify Premier工具完成。然而，市面上存在多种综合工具，例如Vivado和Quartus内置的工具，以及Xilinx和Intel提供的免费设计工具，用于将HDL综合至现场可编程门阵列（参见第5.6.2节）。

并非所有这些命令都能综合成硬件。例如，在仿真期间将结果打印到屏幕的命令无法转化为硬件。由于我们的主要兴趣是构建硬件，我们将重点放在语言的*synthesizable subset*上。具体来说，我们将HDL代码划分为*synthesizable*模块和*testbench*。可综合模块描述硬件。测试平台包含用于向模块施加输入、检查输出结果是否正确以及打印预期输出与实际输出之间差异的代码。测试平台代码仅用于仿真，无法综合。

初学者最常见的错误之一，是将硬件描述语言（HDL）视为计算机程序，而非描述数字硬件的简写形式。若不清楚HDL应综合出何种硬件，最终结果很可能不尽人意——可能生成远超必要的硬件，或写出能正确仿真却无法在硬件中实现的代码。相反，应从组合逻辑块、寄存器及有限状态机的角度思考系统设计。在编写代码前，先在纸上勾勒这些模块并标注其连接方式。

根据我们的经验，学习HDL的最佳方式是通过实例。HDL有特定的方法来描述各类逻辑电路，这些方法被称为*idioms*。本章将教你如何为每种类型的模块编写正确的HDL惯用写法，然后如何将这些模块组合起来构建一个可工作的系统。当你需要描述特定类型的硬件时，请寻找类似的示例并根据你的需求进行调整。我们不会试图严格定义HDL的所有语法，因为那会极其枯燥，并且容易让人将HDL视为编程语言，而非硬件的简写形式。IEEE SystemVerilog和VHDL规范，以及众多枯燥但详尽的教科书，包含了所有细节——如果你发现自己需要某个特定主题的更多信息，可以参考这些资料。（参见本书末尾的“延伸阅读”部分。）

4.2 组合逻辑

回顾一下，我们要求自己设计同步时序电路，这类电路由组合逻辑和寄存器组成。组合逻辑的输出仅取决于当前输入。本节将介绍如何使用硬件描述语言（HDL）编写组合逻辑的行为模型。

4.2.1 位运算符

*Bitwise*操作符作用于单比特信号或多比特总线。例如，HDL示例4.2中的*inv*模块描述了连接到4位总线的四个反相器。

HDL示例4.2 反相器

SystemVerilog

```
module inv(input  logic [3:0] a,
           output logic [3:0] y);

    assign y = ~a;
endmodule
```

`a[3:0]` represents a 4-bit bus. The bits, from most significant to least significant, are `a[3]`, `a[2]`, `a[1]`, and `a[0]`. This is called *little-endian* order, because the least significant bit has the smallest bit number. We could have named the bus `a[4:1]`, in which case `a[4]` would have been the most significant. Or we could have used `a[0:3]`, in which case the bits, from most significant to least significant, would be `a[0]`, `a[1]`, `a[2]`, and `a[3]`. This is called *big-endian* order.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity inv is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of inv is
begin
    y <= not a;
end;
```

VHDL uses `STD_LOGIC_VECTOR` to indicate busses of `STD_LOGIC`. `STD_LOGIC_VECTOR(3 downto 0)` represents a 4-bit bus. The bits, from most significant to least significant, are `a(3)`, `a(2)`, `a(1)`, and `a(0)`. This is called *little-endian* order, because the least significant bit has the smallest bit number. We could have declared the bus to be `STD_LOGIC_VECTOR(4 downto 1)`, in which case bit 4 would have been the most significant. Or we could have written `STD_LOGIC_VECTOR(0 to 3)`, in which case the bits, from most significant to least significant, would be `a(0)`, `a(1)`, `a(2)`, and `a(3)`. This is called *big-endian* order.

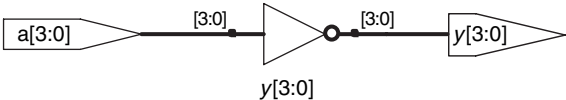


图4.3 inv综合电路

总线的字节序完全是任意的。（关于该术语的起源，请参见第6.6.1节的边栏。）实际上，字节序与此示例也无关，因为反相器组不关心位的顺序。字节序仅对加法等运算符有意义，因为在这些运算中，某一列的求和结果会进位到下一列。只要一致使用，两种顺序都是可接受的。我们将一致使用小端序，在SystemVerilog中表示为[N - 1:0]，在VHDL中表示为(N - 1 downto 0)，用于N位总线。

本章每个代码示例之后，均附有由Synplify Premier综合工具根据SystemVerilog代码生成的原理图。图4.3显示，inv模块综合为一组四个反相器，由标记为y[3:0]的反相器符号表示。该反相器组连接至4位输入和输出总线。综合后的VHDL代码也会生成类似的硬件结构。

HDL示例4.3中的gates模块演示了作用于4位总线的其他基本逻辑函数的按位运算。

HDL示例4.3 逻辑门

SystemVerilog

模块 gates(输入 逻辑 [3:0] a, b, 输出 逻辑 [3:0] y1, y2, y3, y4, y5);

/* 五种不同的双输入逻辑门作用于4位总线 */
assign y1 = a & b; // 与门 assign y2 = a | b; // 或门 assign y3 = a ^ b; // 异或门
assign y4 = ~(a & b); // 与非门 assign y5 = ~(a | b); // 或非门 endmodule

~&|是SystemVerilog operators的示例，而a、b和y1是 operands。运算符和操作数的组合，例如a & b或~(a | b)，称为 expression。像assign y4 = ~(a & b);这样的完整命令称为 statement。

assign out = in1 op in2; 被称为 continuous assignment statement。连续赋值语句以分号结束。每当连续赋值语句中 = 右侧的输入发生变化时，左侧的输出会重新计算。因此，连续赋值语句描述组合逻辑。

VHDL

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity gates is
port(a, b: in STD_LOGIC_VECTOR(3 downto 0);
y1, y2, y3, y4, y5: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of gates is
begin
-- five different two-input logic gates
-- acting on 4-bit busses
y1 <= a and b;
y2 <= a or b;
y3 <= a xor b;
y4 <= a nand b;
y5 <= a nor b;
end;

not、xor、and 和 or 是 VHDL operators 的示例，而 a、b 和 y1 是 operands。运算符和操作数的组合，例如 a and b 或 a nor b，称为 expression。完整的命令，例如 y4 <= a nand b; 称为 statement。

out <= in1 op in2; 被称为 concurrent signal assignment statement。VHDL 赋值语句以分号结束。每当并发信号赋值语句中 <= 右侧的输入发生变化时，左侧的输出会重新计算。因此，并发信号赋值语句描述组合逻辑。

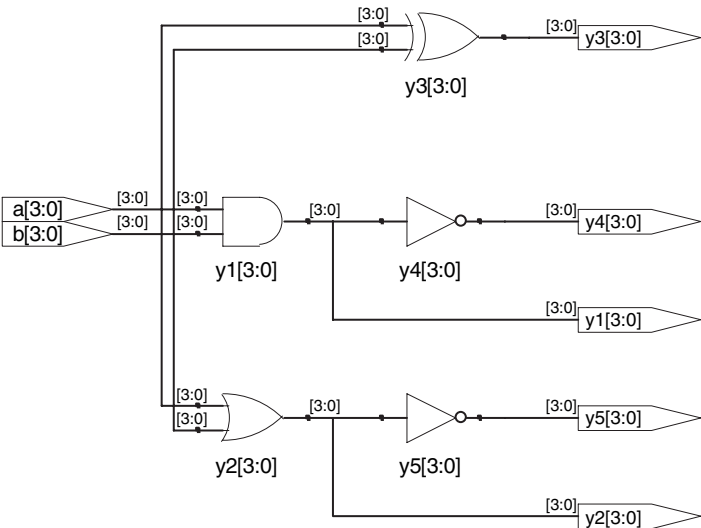


图4.4 门级综合电路

4.2.2 注释和空白

gates示例展示了如何格式化注释。SystemVerilog和VHDL对空白字符（即空格、制表符和换行符）的使用并不严格。然而，适当的缩进和空行的使用有助于使复杂的设计更具可读性。在信号和模块名称中，请保持大小写和下划线使用的一致性。本文档全部使用小写字母。模块和信号名称不能以数字开头。

SystemVerilog SystemVerilog 注释与 C 或 Java 中的注释完全相同。以 /* 开头的注释会继续，可能跨越多行，直到下一个 */。以 // 开头的注释会继续到行尾。 SystemVerilog 是区分大小写的。y1 和 Y1 在 SystemVerilog 中是不同的信号。然而，使用仅大小写不同的多个信号会让人困惑。	VHDL 以 /* 开头的注释会继续，可能跨越多行，直到遇到 */ 为止。以 -- 开头的注释会继续到行尾。 VHDL 不区分大小写。在 VHDL 中，y1 和 Y1 是同一个信号。然而，其他可能读取你文件的工具可能会区分大小写，如果你随意混用大小写，会导致难以发现的错误。
--	---

4.2.3 归约操作符

归约运算符表示作用于单条总线的多输入门。HDL 示例 4.4 描述了一个八输入与门，其输入为 a₇、a₆、...、a₀。类似的归约运算符也存在于或门、异或门、与非门、或非门和同或门中。回顾一下，多输入异或门执行奇偶校验，当奇数个输入为真时返回真。

HDL 示例 4.4 八输入与门

SystemVerilog <pre>module and8(input logic [7:0] a, output logic y); assign y=&a; // &a is much easier to write than // assign y = a[7] & a[6] & a[5] & a[4] & // a[3] & a[2] & a[1] & a[0]; endmodule</pre>	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity and8 is port(a: in STD_LOGIC_VECTOR(7 downto 0); y: out STD_LOGIC); end; architecture synth of and8 is begin y <= and a; -- and a is much easier to write than -- y <= a(7) and a(6) and a(5) and a(4) and -- a(3) and a(2) and a(1) and a(0); end;</pre>
--	--

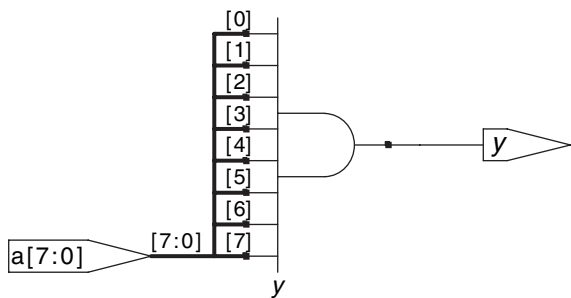


图4.5 和8综合电路

4.2.4 条件赋值

*Conditional assignments*根据输入*condition*从备选信号中选择输出。HD L示例4.5展示了使用条件赋值实现的2:1多路选择器。

HDL示例4.5 2:1多路选择器

SystemVerilog	VHDL
<i>conditional operator</i> ?: 根据第一个表达式在第二个和第三个表达式之间进行选择。第一个表达式称为 <i>condition</i>。如果条件为 1，运算符选择第二个表达式。如果条件为 0，运算符选择第三个表达式。 ?: 对于描述多路选择器特别有用，因为它根据第一个输入在两个其他输入之间进行选择。以下代码展示了使用条件运算符实现具有4位输入和输出的2:1多路选择器的惯用写法。 <pre>模块 mux2(输入 逻辑 [3:0] d0, d1, 输入 逻辑 s, 输出 逻辑 [3:0] y); assign y = s ? d1 : d0; endmodul e</pre> 如果 s 为 1，则 y = d1。如果 s 为 0，则 y = d0。 ?: 也被称为 <i>ternary operator</i>，因为它接受三个输入。在 C 和 Java 编程语言中，它用于相同的目的。	<i>Conditional signal assignments</i>根据某些条件执行不同的操作。它们对于描述多路选择器特别有用。例如，一个2:1多路选择器可以使用条件信号赋值来选择两个4位输入中的一个。 库 IEEE; 使用 IEEE.STD_LOGIC_1164.全部; 实体 mux2 的端口为(d0, d1: 输入 STD_LOGIC_VECTOR(3 downto 0); s: 输入 STD_LOGIC; y: 输出 STD_LOGIC_VECTOR(3 downto 0)); 结束; 架构综合 mux2 是 begin y <= d1 当 s 否则 d0; end; 条件信号赋值在s为1时将y设为d1，否则将y设为d0。注意，在2008年修订VHDL之前，必须写成when s = '1'而非when s。

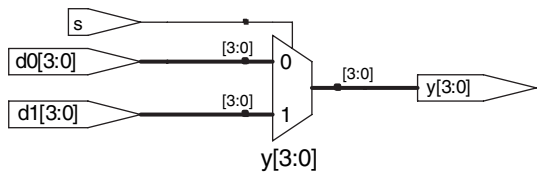


图4.6 mux2综合电路

HDL示例4.6展示了一个基于与HDL示例4.5中2:1多路复用器相同原理的4:1多路复用器。图4.7显示了由综合工具生成的4:1多路复用器原理图。该软件使用了与本文此前所示不同的多路复用器符号。该多路复用器具有多个数据(d)输入和单热使能(e)输入。当其中一个使能信号被置位时，关联的数据被传递至输出。例如，当s[1] = s[0] = 为0时，底部的与门un1_s_5产生1，使能多路复用器的底部输入，从而使其选择d0[3:0]。

HDL示例4.6 4:1多路选择器

SystemVerilog

A 4:1 multiplexer can select one of four inputs, using nested conditional operators.

```
module mux4(input  logic [3:0] d0, d1, d2, d3,
            input  logic [1:0] s,
            output logic [3:0] y);

    assign y = s[1] ? (s[0] ? d3 : d2)
                : (s[0] ? d1 : d0);
endmodule
```

If s[1] is 1, then the multiplexer chooses the first expression, (s[0] ? d3 : d2). This expression, in turn, chooses either d3 or d2 based on s[0] (y = d3 if s[0] is 1 and d2 if s[0] is 0). If s[1] is 0, then the multiplexer similarly chooses the second expression, which gives either d1 or d0 based on s[0].

VHDL

A 4:1 multiplexer can select one of four inputs, using multiple else clauses in the conditional signal assignment.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1,
          d2, d3: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC_VECTOR(1 downto 0);
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth1 of mux4 is
begin
    y <= d0 when s = "00" else
          d1 when s = "01" else
          d2 when s = "10" else
          d3;
end;
```

VHDL also supports *selected signal assignment statements* to provide a shorthand when selecting from one of several possibilities. This is analogous to using a switch/case statement in place of multiple if/else statements in some programming languages. The 4:1 multiplexer can be rewritten with selected signal assignment as follows:

```
architecture synth2 of mux4 is
begin
    with s select y <=
        d0 when "00",
        d1 when "01",
        d2 when "10",
        d3 when others;
end;
```

4.2.5 内部变量

通常将复杂函数分解为中间步骤是很方便的。例如，全加器（将在5.2.1节中描述），

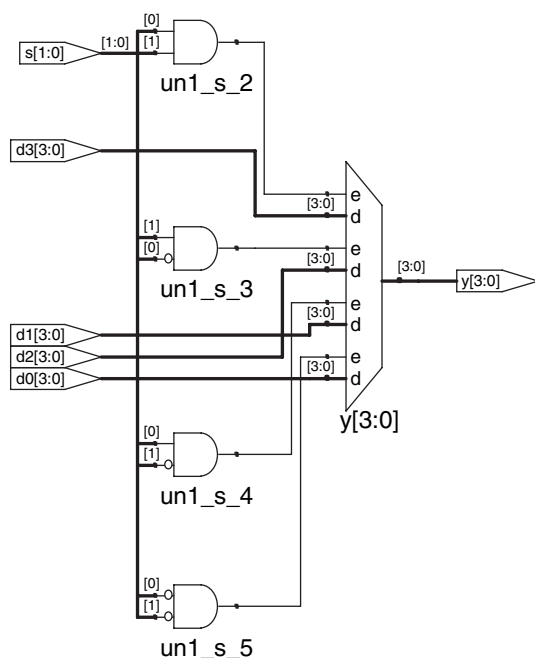


图4.7 mux4综合电路

是一个具有三个输入和两个输出的电路，由以下方程定义：

$$\begin{aligned} S &= A \oplus B \oplus C_{in} \\ C_{out} &= AB + AC_{in} + BC_{in} \end{aligned} \quad (4.1)$$

如果我们定义中间信号， P 和 G ，

$$\begin{aligned} P &= A \oplus B \\ G &= AB \end{aligned} \quad (4.2)$$

我们可以将全加器重写如下：

$$\begin{aligned} S &= P \oplus C_{in} \\ C_{out} &= G + PC_{in} \end{aligned} \quad (4.3)$$

PG 和 $internal\ variables$ 被称为 $internal\ variables$ ，因为它们既不是输入也不是输出；相反，它们仅在模块内部使用。它们类似于编程语言中的局部变量。HDL示例4.7展示了它们在HDL中的使用方式。

HDL 赋值语句（SystemVerilog 中的 `assign` 和 VHDL 中的 `<=`）是并发执行的。这与传统编程语言（如 C 或 Java）不同，在传统语言中，语句按照编写的顺序依次求值。在传统语言中，

通过填写真值表来验证这一点，以确信其正确性。

HDL示例4.7 全加器

SystemVerilog	VHDL
In SystemVerilog, internal signals are usually declared as logic. <pre>module fulladder(input logic a, b, cin, output logic s, cout); logic p, g; assign p = a ^ b; assign g = a & b; assign s = p ^ cin; assign cout = g (p & cin); endmodule</pre>	In VHDL, <i>signals</i> are used to represent internal variables whose values are defined by <i>concurrent signal assignment statements</i>, such as <code>p <= a xor b</code>; <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity fulladder is port(a, b, cin: in STD_LOGIC; s, cout: out STD_LOGIC); end; architecture synth of fulladder is signal p, g: STD_LOGIC; begin p <= a xor b; g <= a and b; s <= p xor cin; cout <= g or (p and cin); end;</pre>

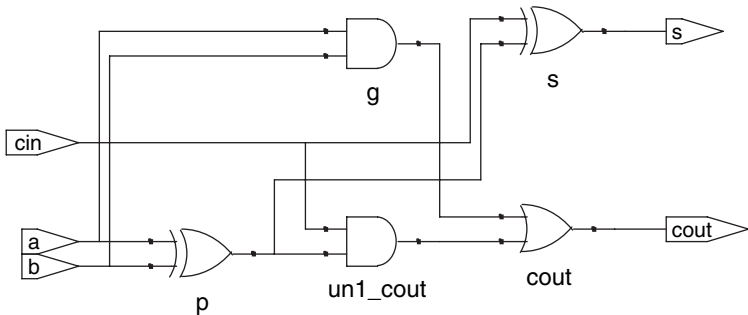


图4.8 全加器综合电路

重要的一点是， $S = P \oplus C_{in}$ 必须出现在 $P = A \oplus B$ 之后，因为语句是按顺序执行的。在硬件描述语言（HDL）中，顺序并不重要。与硬件类似，HDL赋值语句会在其右侧的输入信号值发生变化时被求值，无论这些赋值语句在模块中出现的顺序如何。

4.2.6 优先级

注意，我们在HDL示例4.7中对cout计算使用了括号，以将操作顺序定义为 $C_{out} = G + (P \cdot C_{in})$ ，而不是 $C_{out} = (G + P) \cdot C_{in}$ 。如果不使用括号，默认的操作顺序将由语言定义。HDL示例4.8

HDL示例4.8 运算符优先级

SystemVerilog	VHDL																																								
Table 4.1 SystemVerilog operator precedence <table><tr><th>Op</th><th>Meaning</th></tr><tr><td>H i g h e s t</td><td>~ NOT</td></tr><tr><td></td><td>*, /, % MUL, DIV, MOD</td></tr><tr><td></td><td>+, - PLUS, MINUS</td></tr><tr><td></td><td><<, >> Logical Left/Right Shift</td></tr><tr><td></td><td><<<, >>> Arithmetic Left/Right Shift</td></tr><tr><td></td><td><, <=, >, >= Relative Comparison</td></tr><tr><td></td><td>==, != Equality Comparison</td></tr><tr><td>L o w e s t</td><td>&, ~& AND, NAND</td></tr><tr><td></td><td>^, ~^ XOR, XNOR</td></tr><tr><td></td><td> , ~ OR, NOR</td></tr><tr><td></td><td>?: Conditional</td></tr></table> SystemVerilog 的运算符优先级与其他编程语言中的预期非常相似。特别是，AND 的优先级高于 OR。我们可以利用这一优先级来省略括号： 分配 <code>cout = g p & cin;</code>	Op	Meaning	H i g h e s t	~ NOT		*, /, % MUL, DIV, MOD		+, - PLUS, MINUS		<<, >> Logical Left/Right Shift		<<<, >>> Arithmetic Left/Right Shift		<, <=, >, >= Relative Comparison		==, != Equality Comparison	L o w e s t	&, ~& AND, NAND		^, ~^ XOR, XNOR		, ~ OR, NOR		?: Conditional	Table 4.2 VHDL operator precedence <table><tr><th>Op</th><th>Meaning</th></tr><tr><td>H i g h e s t</td><td>not NOT</td></tr><tr><td></td><td>*, /, mod, rem MUL, DIV, MOD, REM</td></tr><tr><td></td><td>+, - PLUS, MINUS</td></tr><tr><td></td><td>rol, ror, srl, sll Rotate, Shift logical</td></tr><tr><td>L o w e s t</td><td><, <=, >, >= Relative Comparison</td></tr><tr><td></td><td>=, /= Equality Comparison</td></tr><tr><td></td><td>and, or, nand, nor, xor, xnor Logical Operations</td></tr></table> Multiplication has precedence over addition in VHDL, as 你可能会预期。然而，与SystemVerilog不同，所有逻辑运算（与、或等）具有相同的优先级，这与布尔代数中的预期不同。因此，括号是必要的。否则，<code>cout <= g or p and cin</code> 将从左到右被解释为 <code>cout <= (g or p) and cin</code>。	Op	Meaning	H i g h e s t	not NOT		*, /, mod, rem MUL, DIV, MOD, REM		+, - PLUS, MINUS		rol, ror, srl, sll Rotate, Shift logical	L o w e s t	<, <=, >, >= Relative Comparison		=, /= Equality Comparison		and, or, nand, nor, xor, xnor Logical Operations
Op	Meaning																																								
H i g h e s t	~ NOT																																								
	*, /, % MUL, DIV, MOD																																								
	+, - PLUS, MINUS																																								
	<<, >> Logical Left/Right Shift																																								
	<<<, >>> Arithmetic Left/Right Shift																																								
	<, <=, >, >= Relative Comparison																																								
	==, != Equality Comparison																																								
L o w e s t	&, ~& AND, NAND																																								
	^, ~^ XOR, XNOR																																								
	, ~ OR, NOR																																								
	?: Conditional																																								
Op	Meaning																																								
H i g h e s t	not NOT																																								
	*, /, mod, rem MUL, DIV, MOD, REM																																								
	+, - PLUS, MINUS																																								
	rol, ror, srl, sll Rotate, Shift logical																																								
L o w e s t	<, <=, >, >= Relative Comparison																																								
	=, /= Equality Comparison																																								
	and, or, nand, nor, xor, xnor Logical Operations																																								

指定每种语言中运算符的优先级从高到低。表格中包括将在第5章定义的算术、移位和比较运算符。

4.2.7 数字

数字可以用二进制、八进制、十进制或十六进制（分别对应基数2、8、10和16）表示。可以可选地指定大小，即位数，并会插入前导零以达到该大小。数字中的下划线会被忽略，有助于将长数字分割成更易读的片段。HDL示例4.9解释了每种语言中数字的书写方式。

HDL示例4.9 数字

SystemVerilog

声明常量的格式为N'Bvalue，其中N是以比特为单位的位宽，B是表示进制的字母，value给出具体数值。例如，9'h25表示一个9位数字，其值为 $25_{16} = 37_{10} = 000100101_2$ 。SystemVerilog支持'b表示二进制、'o表示八进制、'd表示十进制、'h表示十六进制。

如果省略基数，则默认为十进制。如果未指定位宽，则假定该数字具有其所在表达式的位宽。数字前面会自动补零以达到完整位宽。例如，若w是一个6位总线，assign w = 'b11会使w的值为000011。显式指定位宽是更好的做法。例外情况是，'0 和 '1 是SystemVerilog中分别用0和1填充所有位的惯用写法。

表4.3 SystemVerilog数字

Numbers	Bits	Base	Val	Stored
3'b101	3	2	5	101
'b11	?	2	3	000 ... 0011
8'b11	8	2	3	00000011
8'b1010_1011	8	2	171	10101011
3'd6	3	10	6	110
6'o42	6	8	34	100010
8'hAB	8	16	171	10101011
42	?	10	42	00 ... 0101010

VHDL

在VHDL中，STD_LOGIC数字以二进制形式书写并用单引号括起来：'0和'1表示逻辑0和1。声明STD_LOGIC_VECTOR常量的格式为NB"value"，其中N是位数，B是表示基数的字母，value给出数值。例如，9X"25"表示一个9位数字，其值为 $25_{16} = 37_{10} = 000100101_2$ 。VHDL 2008支持B表示二进制、O表示八进制、D表示十进制、X表示十六进制。

若省略基数，则默认为二进制。若未指定位宽，则假定数值的位宽与值中指定的位数匹配。例如，y <= X"7B" 要求 y 为 8 位信号。若 y 的位数更多，VHDL 不会 *not* 在左侧用 0 填充该数值，而是在编译期间报错。others => '0' 和 others => '1' 是 VHDL 中分别用 0 和 1 填充所有位的惯用写法。

表4.4 VHDL数字

Numbers	Bits	Base	Val	Stored
3B"101"	3	2	5	101
B"11"	2	2	3	11
8B"11"	8	2	3	00000011
8B"1010_1011"	8	2	171	10101011
3D"6"	3	10	6	110
6O"42"	6	8	34	100010
8X"AB"	8	16	171	10101011
"101"	3	2	5	101
B"101"	3	2	5	101
X"AB"	8	16	171	10101011

4.2.8 Z和X

HDL使用z表示浮空值。z在描述三态缓冲器时特别有用，当使能信号为0时，其输出呈浮空状态。回顾第2.6.2节，总线可由多个三态缓冲器驱动，且恰好应有一个缓冲器被使能。HDL示例4.10展示了一个三态缓冲器的惯用写法。若缓冲器被使能，输出与输入相同；若缓冲器被禁用，则输出被赋值为浮空值（z）。

类似地，HDL使用x表示无效的逻辑电平。如果一条总线被两个使能的三态缓冲器（或其他门电路）同时驱动为0和1，结果即为x，表示竞争。如果驱动总线的所有三态缓冲器同时关闭，总线将处于高阻态，用z表示。

在仿真开始时，状态节点（如触发器输出）被初始化为未知状态（SystemVerilog中为x，VHDL中为u）。这有助于追踪因在触发器输出被使用前忘记复位而导致的错误。

HDL示例4.10 三态缓冲器

SystemVerilog

模块三态（输入 逻辑 [3:0] a，输入 逻辑 en，输出 三态 [3:0] y）；

assign y = en ? a : 4'bz; endmodule

请注意，y被声明为tri而非logic。logic信号只能有一个驱动源。三态总线可以有多个驱动源，因此应将其声明为net。SystemVerilog中有两种类型的网络称为tri和triereg。通常，网络上的一个驱动源在某一时刻处于活动状态，网络采用该值。如果没有驱动源处于活动状态，tri会浮空（z），而triereg会保持之前的值。如果输入或输出未指定类型，则默认为tri。此外，请注意，模块的tri输出可以用作另一个模块的logic输入。第4.7节将进一步讨论具有多个驱动源的网络。

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity tristate is
  port(a: in STD_LOGIC_VECTOR(3 downto 0);
       en: in STD_LOGIC;
       y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of tristate is
begin
  y <= a when en else "ZZZZ";
end;
```

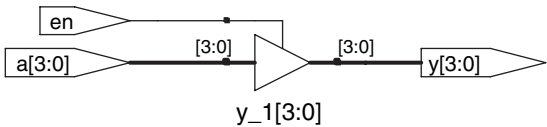


图4.9 三态合成电路

如果门电路接收到浮空输入，当无法确定正确的输出值时，可能会产生x输出。同样，如果接收到非法或未初始化的输入，也可能产生x输出。HDL示例4.11

HDL示例4.11 包含未定义和浮动输入的真值表

SystemVerilog

SystemVerilog 信号值为 0、1、z 和 x。以 z 或 x 开头的 SystemVerilog 常量在必要时会用前导 z 或 x（而不是 0）填充以达到其完整长度。

表4.5展示了使用所有四种可能信号值的与门真值表。请注意，即使某些输入未知，该门有时也能确定输出。例如，0 & z 返回0，因为如果任一输入为0，与门的输出始终为0。否则，浮空或无效输入会导致无效输出，在SystemVerilog中显示为x。

VHDL

VHDL STD_LOGIC信号包括'0'、'1'、'z'、'x'和'u'。

表4.6展示了使用所有五种可能信号值的与门真值表。请注意，即使某些输入未知，该门有时也能确定输出。例如，'0'和'z'返回'0'，因为如果任一输入为'0'，与门的输出始终为'0'。否则，浮空或无效输入会导致无效输出，在VHDL中显示为'x'。未初始化的输入会导致未初始化的输出，在VHDL中显示为'u'。

表4.5 带z和x的SystemVerilog与门真值表

&		A			
		0	1	z	x
B	0	0	0	0	0
	1	0	1	x	x
	z	0	x	x	x
	x	0	x	x	x

表4.6 含z、x和u的VHDL与门真值表

AND		A				
		0	1	z	x	u
B	0	0	0	0	0	0
	1	0	1	x	x	u
	z	0	x	x	x	u
	x	0	x	x	x	u
	u	0	u	u	u	u

HDL示例4.12 位操作

SystemVerilog 分配 <code>y = {c[2:1], {3{d[0]}}, c[0], 3'b101};</code> <code>{}</code> 运算符用于连接总线。<code>{3{d[0]}}</code> 表示 <code>d[0]</code> 的三份副本。 不要将3位二进制常量<code>3'b101</code>与名为<code>b</code>的总线混淆。注意，在常量中指定3位的长度至关重要。否则，它会有未知数量的前导零，这些零可能会出现在<code>y</code>的中间。 如果 <code>y</code> 的宽度超过 9 位，则会在最高有效位补零。	VHDL <code>y <= (c(2 downto 1), d(0), d(0), d(0), c(0), 3B"101");</code> <code>()</code> 聚合运算符用于连接总线。<code>y</code> 必须是一个 9 位的 <code>STD_LOGIC_VECTOR</code>。 另一个示例展示了VHDL聚合的强大功能。假设<code>z</code>是一个8位<code>STD_LOGIC_VECTOR</code>，通过以下命令聚合将<code>z</code>赋值为10010110。 <code>z <= ("10", 4 => '1', 2 downto 1 => '1', others => '0')</code> "10" 位于前导位对中。1 也被置于位 4 以及位 2 和位 1 中。其他位为 0。
--	--

展示了SystemVerilog和VHDL如何在逻辑门中组合这些不同的信号值{v*}。

在仿真中看到 `x` 或 `u` 值几乎总是表明存在错误或不良编码习惯。在综合电路中，这对应着浮空栅极输入、未初始化状态或竞争。`x` 或 `u` 可能被电路随机解释为 0 或 1，从而导致不可预测的行为。

4.2.9 位混合

通常需要对总线的子集进行操作，或者将信号连接（拼接）起来形成总线。这些操作统称为`bit swizzling`。在HDL示例4.12中，通过位拼接操作，`y`被赋值为9位数值`c2c1d0d0d0c0101`。

4.2.10 延迟

HDL 语句可以与以任意单位指定的延迟相关联。在仿真过程中，它们有助于预测电路的工作速度（如果指定了有意义的延迟），并可用于调试目的。

理解因果关系（如果仿真结果中所有信号同时变化，则很难推断不良输出的来源）。这些延迟在综合过程中被忽略；综合器产生的门延迟取决于其 t_{pd} 和 t_{cd} 规格，而非HDL代码中的数字。

HDL示例4.13在HDL示例4.1的原始函数 $y = a \oplus b \oplus c$ 中增加了延迟。它假设反相器的延迟为1纳秒，三输入与门的延迟为2纳秒，三输入或门的延迟为4纳秒。图4.10显示了仿真波形，其中 y 在输入后延迟7纳秒。请注意，在仿真开始时 y 初始状态为未知。

HDL示例4.13 带延迟的逻辑门

SystemVerilog	VHDL
<pre>'timescale 1ns/1ps module example(input logic a, b, c, output logic y); logic ab, bb, cb, n1, n2, n3; assign #1 {ab, bb, cb} = ~(a, b, c); assign #2 n1 = ab & bb & cb; assign #2 n2 = a & bb & cb; assign #2 n3 = a & bb & c; assign #4 y = n1 n2 n3; endmodule</pre> SystemVerilog files can include a timescale directive that indicates the value of each time unit. The statement is of the form <code>'timescale unit/precision</code>. In this file, each unit is 1 ns and the simulation has 1 ps precision. If no timescale directive is given in the file, a default unit and precision (usually 1 ns for both) are used. In SystemVerilog, a <code>#</code> symbol is used to indicate the number of units of delay. It can be placed in assign statements, as well as nonblocking (<code><=</code>) and blocking (<code>=</code>) assignments, which will be discussed in Section 4.5.4.	<pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity example is port(a, b, c: in STD_LOGIC; y: out STD_LOGIC); end; architecture synth of example is signal ab, bb, cb, n1, n2, n3: STD_LOGIC; begin ab <= not a after 1 ns; bb <= not b after 1 ns; cb <= not c after 1 ns; n1 <= ab and bb and cb after 2 ns; n2 <= a and bb and cb after 2 ns; n3 <= a and bb and c after 2 ns; y <= n1 or n2 or n3 after 4 ns; end;</pre> In VHDL, the <code>after</code> clause is used to indicate delay. The units, in this case, are specified as nanoseconds.

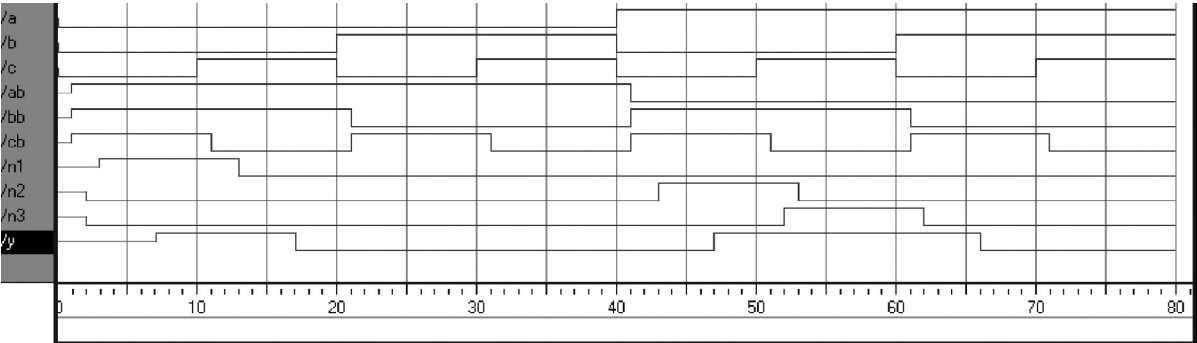


图4.10 带延迟的示例仿真波形（来自ModelSim仿真器）

4.3 结构建模

上一节讨论了*behavioral*建模，根据输入与输出之间的关系描述模块。本节探讨*structural*建模，根据模块如何由更简单的模块组成来描述模块。

例如，HDL示例4.14展示了如何用三个2:1多路选择器组装成一个4:1多路选择器。每个2:1多路选择器的副本被称为

HDL示例4.14 4:1多路选择器的结构模型

SystemVerilog

```
module mux4(input logic [3:0] d0, d1, d2, d3,
            input logic [1:0] s,
            output logic [3:0] y);

    logic [3:0] low, high;

    mux2 lowmux(d0, d1, s[0], low);
    mux2 highmux(d2, d3, s[0], high);
    mux2 finalmux(low, high, s[1], y);
endmodule
```

The three mux2 instances are called lowmux, highmux, and finalmux. The mux2 module must be defined elsewhere in the SystemVerilog code—see HDL Example 4.5, 4.15, or 4.34.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1,
          d2, d3: in STD_LOGIC_VECTOR(3 downto 0);
          s: in STD_LOGIC_VECTOR(1 downto 0);
          y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture struct of mux4 is
    component mux2
        port(d0,
              d1: in STD_LOGIC_VECTOR(3 downto 0);
              s: in STD_LOGIC;
              y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    signal low, high: STD_LOGIC_VECTOR(3 downto 0);
begin
    lowmux: mux2 port map(d0, d1, s(0), low);
    highmux: mux2 port map(d2, d3, s(0), high);
    finalmux: mux2 port map(low, high, s(1), y);
end;
```

The architecture must first declare the mux2 ports using the component declaration statement. This allows VHDL tools to check that the component you wish to use has the same ports as the entity that was declared somewhere else in another entity statement, preventing errors caused by changing the entity but not the instance. However, component declaration makes VHDL code rather cumbersome.

Note that this architecture of mux4 was named *struct*, whereas architectures of modules with behavioral descriptions from Section 4.2 were named *synth*. VHDL allows multiple architectures (implementations) for the same entity; the architectures are distinguished by name. The names themselves have no significance to the CAD tools, but *struct* and *synth* are common. Synthesizable VHDL code generally contains only one architecture for each entity, so we will not discuss the VHDL syntax to configure which architecture is used when multiple architectures are defined.

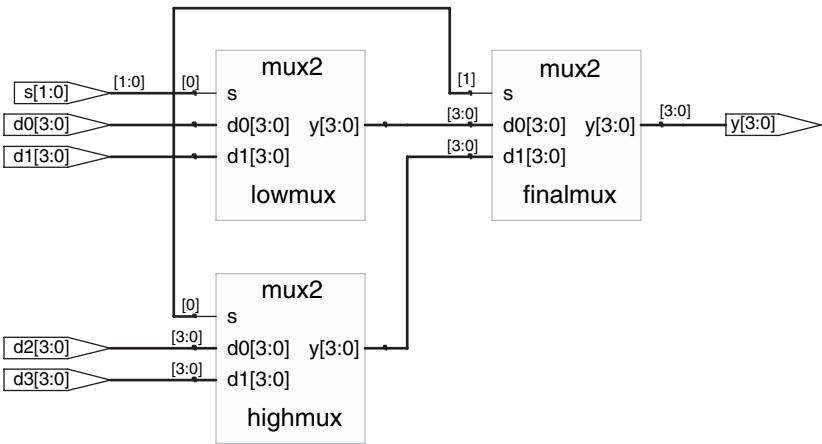


图4.11 mux4综合电路

一个*instance*。同一模块的多个实例通过不同的名称区分，本例中为lowmux、highmux和finalmux。这是规则性的一个示例，其中2:1多路复用器被多次重用。

HDL示例4.15使用结构建模，通过一对三态缓冲器构建了一个2:1多路选择器。然而，不建议使用三态器件构建逻辑电路。

HDL示例4.15 2:1多路选择器的结构模型

SystemVerilog <pre>module mux2(input logic [3:0] d0, d1, input logic s, output tri [3:0] y); tristate t0(d0, ~s, y); tristate t1(d1, s, y); endmodule</pre> In SystemVerilog, expressions such as <code>~s</code> are permitted in the port list for an instance. Arbitrarily complicated expressions are legal but discouraged because they make the code difficult to read.	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity mux2 is port(d0, d1: in STD_LOGIC_VECTOR(3 downto 0); s: in STD_LOGIC; y: out STD_LOGIC_VECTOR(3 downto 0)); end; architecture struct of mux2 is component tristate port(a: in STD_LOGIC_VECTOR(3 downto 0); en: in STD_LOGIC; y: out STD_LOGIC_VECTOR(3 downto 0)); end component; signal sbar: STD_LOGIC; begin sbar <= not s; t0: tristate port map(d0, sbar, y); t1: tristate port map(d1, s, y); end;</pre> In VHDL, expressions such as <code>not s</code> are not permitted in the port map for an instance. Thus, <code>sbar</code> must be defined as a separate signal.
---	---

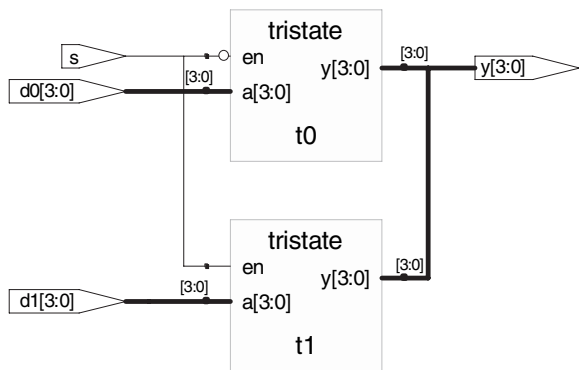


图4.12 mux2综合电路

HDL示例4.16展示了模块如何访问总线的一部分。一个8位宽的2:1多路复用器由两个已定义的4位2:1多路复用器构建而成，分别操作字节的低半字节和高半字节。

一般来说，复杂系统的设计遵循 *hierarchically* 的规则。整个系统通过实例化其主要组件来进行结构性描述。每个组件又进一步从其构建模块开始进行结构性描述，如此递归进行，直至各个部分简单到足以进行行为性描述。良好的风格是避免（或至少尽量减少）在单个模块中混合结构性和行为性描述。

HDL示例4.16 访问总线部分

SystemVerilog	VHDL
<pre>module mux2_8(input logic [7:0] d0, d1, input logic s, output logic [7:0] y); mux2 lsbmux (d0[3:0], d1[3:0], s, y[3:0]); mux2 msbmux (d0[7:4], d1[7:4], s, y[7:4]); endmodule</pre>	<pre>VHDL library IEEE; use IEEE.STD_LOGIC_1164.all; entity mux2_8 is port(d0, d1: in STD_LOGIC_VECTOR(7 downto 0); s: in STD_LOGIC; y: out STD_LOGIC_VECTOR(7 downto 0)); end; architecture struct of mux2_8 is component mux2 port(d0, d1: in STD_LOGIC_VECTOR(3 downto 0); s: in STD_LOGIC; y: out STD_LOGIC_VECTOR(3 downto 0)); end component; begin lsbmux: mux2 port map(d0(3 downto 0), d1(3 downto 0), s, y(3 downto 0)); msbmux: mux2 port map(d0(7 downto 4), d1(7 downto 4), s, y(7 downto 4)); end;</pre>

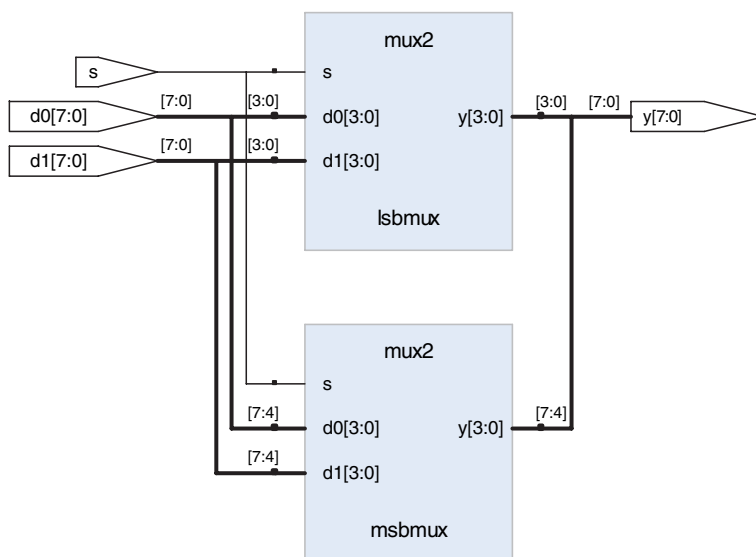


图4.13 mux2_8综合电路

4.4 时序逻辑

HDL 综合器能够识别特定的代码模式，并将其转化为具体的时序电路。其他编码风格可能仿真正确，但综合出的电路会存在明显或微妙的错误。本节介绍描述寄存器和锁存器的正确代码模式。

4.4.1 寄存器

绝大多数现代商业系统都使用正边沿触发的D触发器构建寄存器。HDL示例4.17展示了此类触发器的惯用写法。

在SystemVerilog的always语句和VHDL的process语句中，信号会保持其旧值，直到敏感列表中的某个事件发生并显式导致其改变。因此，通过适当的敏感列表，此类代码可用于描述具有存储功能的时序电路。例如，触发器仅在敏感列表中包含clk。它会记住q的旧值，直到下一个clk上升沿，即使在此期间d发生变化。

相比之下，SystemVerilog连续赋值语句（assign）和VHDL并发赋值语句（<=）会在右侧任一输入发生变化时重新求值。因此，此类代码必然描述组合逻辑。

HDL示例4.17 寄存器

SystemVerilog

```
模块 flop(输入 逻辑 clk, 输入 逻辑 [3:0] d,
输出 逻辑 [3:0] q); always_ff @(posedge clk)
q <= d; endmodule
```

通常，SystemVerilog 的 always 语句写成如下形式

```
always @(敏感列表) 语句;
```

该语句仅在敏感列表中指定的事件发生时执行。在此示例中，语句为 q <= d（读作“q 得到 d”）。因此，触发器在时钟上升沿将 d 复制到 q，否则保持 q 的旧状态。请注意，敏感列表也称为激励列表。

<=被称为*nonblocking assignment*。目前可以将其视为一个普通的=符号；我们将在第4.5.4节中讨论更细微的要点。请注意，在always语句内部使用<=而非assign。

如后续章节所示，always语句可根据敏感列表和语句内容用于推断触发器、锁存器或组合逻辑。由于这种灵活性，很容易无意中产生错误的硬件。SystemVerilog引入了always_ff、always_latch和always_comb来降低常见错误的风险。always_ff的行为与always类似，但专门用于推断触发器，并在推断出其他逻辑时允许工具发出警告。

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity flop is
  port(clk: in STD_LOGIC;
        d: in STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of flop is
begin
  process(clk) begin
    if rising_edge(clk) then
      q <= d;
    end if;
  end process;
end;
```

VHDL进程的编写形式如下

```
process(敏感列表) begin 语句; end proces
s;
```

当敏感列表中的任何变量发生变化时，该语句被执行。在此示例中，if语句检查变化是否为clk的上升沿。如果是，则q <= d（读作“q得到d”）。因此，触发器在时钟正沿将d复制到q，否则保持q的旧状态。

触发器的另一种VHDL惯用写法是

```
process(clk) begin  if clk'event and clk = '1'
then  q <= d;  end if; end process;
```

rising_edge(clk) 与 clk'event 和 clk = '1' 同义。

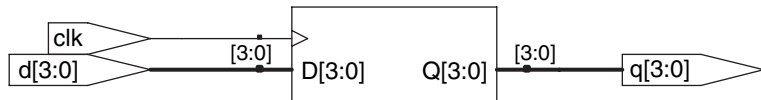


图4.14 flop综合电路

4.4.2 可重置寄存器

当仿真开始或电路首次上电时，触发器或寄存器的输出处于未知状态。在SystemVerilog中用x表示，在VHDL中用u表示。通常，最佳实践是使用可复位的寄存器，以便在上电时使系统进入已知状态。复位可以是异步的，也可以是同步的。请注意，异步复位会立即生效，而同步复位仅在时钟沿到来时清除输出。

时钟的下一个上升沿。HDL示例4.18展示了具有异步复位和同步复位的触发器的惯用写法。请注意，在原理图中区分同步复位和异步复位可能较为困难。

HDL示例4.18 可复位寄存器

SystemVerilog

```
module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

// asynchronous reset
always_ff @(posedge clk, posedge reset)
    if (reset) q <= 4'b0;
    else      q <= d;
endmodule

module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

// synchronous reset
always_ff @(posedge clk)
    if (reset) q <= 4'b0;
    else      q <= d;
endmodule
```

在always语句的敏感列表中，多个信号用逗号或“or”分隔。请注意，posedge reset出现在异步复位触发器的敏感列表中，但不在同步复位触发器的敏感列表中。因此，异步复位触发器会立即响应reset的上升沿，而同步复位触发器仅在时钟上升沿响应reset。

由于模块名称相同（flopr），您在设计时只能包含其中一个。若需同时使用两者，则必须重命名其中一个模块。

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture asynchronous of flopr is
begin
    process(clk, reset) begin
        if reset then
            q <= "0000";
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synchronous of flopr is
begin
    process(clk) begin
        if rising_edge(clk) then
            if reset then q <= "0000";
            else q <= d;
            end if;
        end if;
    end process;
end;
```

进程敏感列表中的多个信号用逗号分隔。请注意，复位信号出现在异步复位触发器的敏感列表中，但不在同步复位触发器的敏感列表中。因此，异步复位触发器会立即响应复位信号的上升沿，而同步复位触发器仅在时钟上升沿时响应复位信号。

如前所述，架构名称（本例中为异步或同步）会被VHDL工具忽略，但可能有助于阅读代码的人理解。由于两种架构都描述了实体flopr，因此在设计中只能包含其中一种。若需同时使用两者，则需重命名其中一个模块。

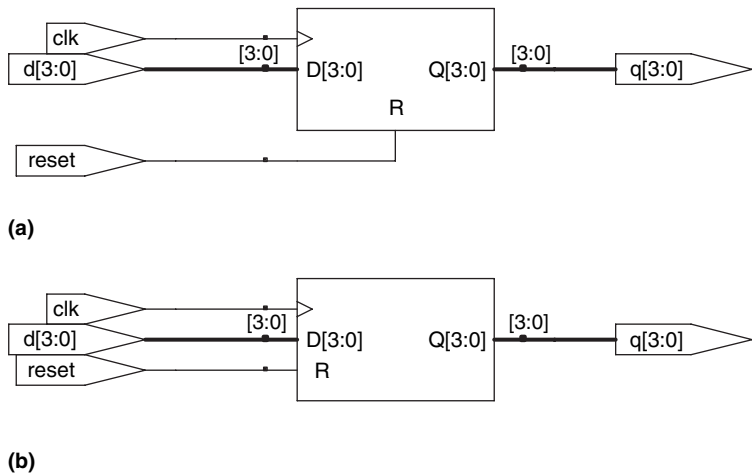


图 图4.15 flopr综合电路 (a) 异步复位, (b) 同步 用户重置

4.4.3 启用寄存器

使能寄存器仅在使能信号有效时响应时钟。HDL示例4.19展示了一个异步可复位的使能寄存器，当复位和使能信号均为假时，该寄存器保持其旧值。

HDL示例4.19 可复位使能寄存器

SystemVerilog <pre>module flopenr(input logic clk, input logic reset, input logic en, input logic [3:0] d, output logic [3:0] q); // asynchronous reset always_ff @(posedge clk, posedge reset) if (reset) q <= 4'b0; else if (en) q <= d; endmodule</pre>	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity flopenr is port(clk, reset, en: in STD_LOGIC; d: in STD_LOGIC_VECTOR(3 downto 0); q: out STD_LOGIC_VECTOR(3 downto 0)); end; architecture asynchronous of flopenr is -- asynchronous reset begin process(clk, reset) begin if reset then q <= "0000"; elsif rising_edge(clk) then if en then q <= d; end if; end if; end process; end;</pre>
--	---

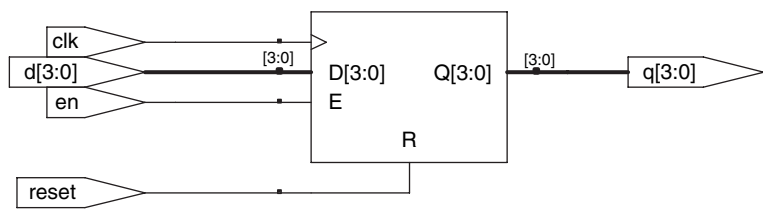


图4.16 flopnr综合电路

4.4.4 多寄存器

一个单一的always/process语句可用于描述多个硬件模块。例如，考虑第3.5.5节中的同步器，它由两个背对背的触发器组成，如图4.17所示。HDL示例4.20描述了该同步器。在clk的上升沿，d被复制到n1。同时，n1被复制到q。

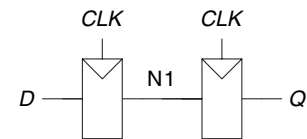


图4.17 同步器电路

HDL示例4.20 同步器

SystemVerilog

```
module sync(input logic clk,
            input logic d,
            output logic q);

    logic n1;

    always_ff @(posedge clk)
    begin
        n1 <= d; // nonblocking
        q <= n1; // nonblocking
    end
endmodule
```

Notice that the begin/end construct is necessary because multiple statements appear in the always statement. This is analogous to {} in C or Java. The begin/end was not needed in the flopnr example because if/else counts as a single statement.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sync is
    port(clk: in STD_LOGIC;
          d: in STD_LOGIC;
          q: out STD_LOGIC);
end;

architecture good of sync is
    signal n1: STD_LOGIC;
begin
    process(clk) begin
        if rising_edge(clk) then
            n1 <= d;
            q <= n1;
        end if;
    end process;
end;
```

n1 must be declared as a signal because it is an internal signal used in the module.

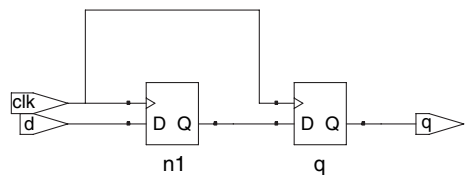


图4.18 同步合成电路

4.4.5 锁存器

回顾第3.2.2节，D锁存器在时钟为高电平时透明，允许数据从输入流向输出。当时钟为低电平时，锁存器变为不透明，保持其旧状态。HDL示例4.21展示了D锁存器的惯用写法。

并非所有综合工具都能很好地支持锁存器。除非你确定你的工具确实支持锁存器，并且有充分的理由使用它们，否则应避免使用锁存器，转而使用边沿触发的触发器。此外，要注意你的HDL描述不应隐含任何非预期的锁存器——若不够细心，很容易出现这种情况。许多综合工具会在创建锁存器时发出警告；若你并未预期出现锁存器，请追踪HDL中的错误。如果你甚至不确定自己是否打算使用锁存器，那么你可能正将HDL当作编程语言来对待，这背后潜藏着更大的问题。

4.5 更多组合逻辑

在第4.2节中，我们使用赋值语句对组合逻辑进行行为描述。SystemVerilog的always语句和VHDL的process

HDL示例4.21 D锁存器

SystemVerilog

```
module latch(input logic clk,
             input logic [3:0] d,
             output logic [3:0] q);

    always_latch
        if (clk) q <= d;
endmodule
```

always_latch is equivalent to always @(clk, d) and is the preferred idiom for describing a latch in SystemVerilog. It evaluates whenever clk or d changes. If clk is HIGH, d flows through to q, so this code describes a positive level-sensitive latch. Otherwise, q keeps its old value. SystemVerilog can generate a warning if the always_latch block doesn't imply a latch.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity latch is
    port(clk: in STD_LOGIC;
          d: in STD_LOGIC_VECTOR(3 downto 0);
          q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of latch is
begin
    process(clk, d) begin
        if clk = '1' then
            q <= d;
        end if;
    end process;
end;
```

The sensitivity list contains both clk and d, so the process evaluates whenever clk or d changes. If clk is HIGH, d flows through to q.

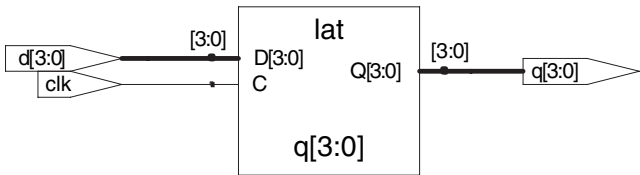


图4.19 锁存器综合电路

HDL示例4.22 使用always/process的反相器

SystemVerilog

```
module inv(input logic [3:0] a,
          output logic [3:0] y);

    always_comb
        y = ~a;
endmodule
```

`always_comb` reevaluates the statements inside the `always` statement whenever any of the signals on the right-hand side of `<=` or `=` in the `always` statement change. In this case, it is equivalent to `always @(a)` but is better because it avoids mistakes if signals in the `always` statement are renamed or added. If the code inside the `always` block is not combinational logic, SystemVerilog will report a warning. `always_comb` is equivalent to `always @(*)` but is preferred in SystemVerilog.

The `=` in the `always` statement is called a *blocking assignment*, in contrast to the `<=` nonblocking assignment. In SystemVerilog, it is good practice to use blocking assignments for combinational logic and nonblocking assignments for sequential logic. This will be discussed further in Section 4.5.4.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity inv is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture proc of inv is
begin
    process(all) begin
        y <= not a;
    end process;
end;
```

`process(all)` reevaluates the statements inside the `process` whenever any of the signals in the `process` change. It is equivalent to `process(a)` but is better because it avoids mistakes if signals in the `process` are renamed or added.

The `begin` and `end` `process` statements are required in VHDL even though the `process` contains only one assignment.

语句用于描述时序电路，因为当未指定新状态时，它们会记住旧状态。然而，如果敏感列表被编写为响应所有输入的变化，并且主体为每种可能的输入组合指定输出值，则`always/process`语句也可用于行为级描述组合逻辑。HDL示例4.22使用`always/process`语句描述了一个由四个反相器组成的阵列（合成电路见图4.3）。

HDL 在 `always/process` 语句中支持 *blocking* 和 *nonblocking assignments*。一组阻塞赋值按照它们在代码中出现的顺序进行评估，正如在标准编程语言中所预期的那样。一组非阻塞赋值则并发评估；所有语句在左侧信号更新之前完成评估。

HDL 示例 4.23 使用中间信号 `p` 和 `g` 来计算 `s` 和 `cout`，从而定义了一个全加器。它生成了与图 4.8 相同的电路，但使用了 `always/process` 语句代替赋值语句。

HDL示例4.22和4.23是使用`always/process`语句建模组合逻辑的糟糕应用，因为它们比HDL示例4.2和4.7中采用赋值语句的等效方法需要更多行代码。然而，`case`和`if`语句在建模更复杂的组合逻辑时更为便捷。`case`和`if`语句必须出现在`always/process`语句内部，将在后续章节中讨论。

SystemVerilog 在SystemVerilog的always语句中，=表示阻塞赋值，<=表示非阻塞赋值（也称为并发赋值）。 不要将这两种类型与连续赋值（即assign语句）混淆。assign语句必须用在always语句之外，并且也是并发求值的。	VHDL 在VHDL进程语句中，:=表示阻塞赋值，<=表示非阻塞赋值（也称为并发赋值）。这是引入:=的第一部分。 非阻塞赋值用于输出和信号。阻塞赋值用于变量，这些变量在过程语句中声明（参见HDL示例4.23）。<=也可以出现在过程语句之外，此时它也是并发求值的。
---	--

HDL示例4.23 使用always/process的全加器

SystemVerilog module fulladder(input logic a, b, cin, output logic s, cout); logic p, g; always_comb begin p = a ^ b; // blocking g = a & b; // blocking s = p ^ cin; // blocking cout = g (p & cin); // blocking end endmodule 在这种情况下，always @(a, b, cin) 原本等同于 always_comb。然而，always_comb 更好，因为它避免了在敏感列表中遗漏信号的常见错误。 由于将在第4.5.4节中讨论的原因，组合逻辑最好使用阻塞赋值。本示例使用阻塞赋值，首先计算p，然后计算g，接着计算s，最后计算cout。	VHDL library IEEE; use IEEE.STD_LOGIC_1164.all; entity fulladder is port(a, b, cin: in STD_LOGIC; s, cout: out STD_LOGIC); end; architecture synth of fulladder is begin process(all) variable p, g: STD_LOGIC; begin p := a xor b; -- blocking g := a 和 b; -- 阻塞 s <= p 异 或 cin; cout <= g 或 (p 与 cin); 结 束进程; 结束; 在这种情况下，process(a, b, cin) 本应等同于 process(all)。然而，process(all) 更好，因为它能避免在敏感列表中遗漏信号的常见错误。 由于将在第4.5.4节中讨论的原因，最好在组合逻辑中对中间变量使用阻塞赋值。本示例对p和g使用了阻塞赋值，以便它们在用于计算依赖于它们的s和cout之前获得新值。 由于p和g出现在进程语句中阻塞赋值（:=）的左侧，因此必须将它们声明为变量而非信号。变量声明应出现在使用该变量的进程中的begin之前。
--	--

4.5.1 Case 语句

在组合逻辑中更好地应用always/process语句的一个例子是七段显示译码器，它利用了必须出现在always/process语句内部的case语句。

正如你在示例2.10的七段数码管译码器中所注意到的，大型组合逻辑模块的设计过程既繁琐又容易出错。硬件描述语言（HDL）带来了极大的改进，允许你在更高的抽象层次上指定功能，然后自动将功能综合为门电路。HDL示例4.24使用case语句基于真值表描述了一个七段数码管译码器。case语句根据其输入的值执行不同的操作。如果所有

HDL示例4.24 七段数码管显示译码器

SystemVerilog <pre>module sevenseg(input logic [3:0] data, output logic [6:0] segments); always_comb case(data) // abc_defg 0: segments = 7'b111_1110; 1: segments = 7'b011_0000; 2: segments = 7'b110_1101; 3: segments = 7'b111_1001; 4: segments = 7'b011_0011; 5: segments = 7'b101_1011; 6: segments = 7'b101_1111; 7: segments = 7'b111_0000; 8: segments = 7'b111_1111; 9: segments = 7'b111_0011; default: segments = 7'b000_0000; endcase endmodule</pre> The case statement checks the value of data. When data is 0, the statement performs the action after the colon, setting segments to 1111110. The case statement similarly checks other data values up to 9 (note the use of the default base, i.e., base 10). 默认子句是一种便捷的方式，用于定义所有未明确列出的情况下的输出，从而保证组合逻辑。 在SystemVerilog中，case语句必须出现在always语句内部。	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity seven_seg_decoder is port(data: in STD_LOGIC_VECTOR(3 downto 0); segments: out STD_LOGIC_VECTOR(6 downto 0)); end; architecture synth of seven_seg_decoder is begin process(all) begin case data is -- abcdefg when X"0" => segments <= "1111110"; when X"1" => segments <= "0110000"; when X"2" => segments <= "1101101"; when X"3" => segments <= "1111001"; when X"4" => segments <= "0110011"; when X"5" => segments <= "1011011"; when X"6" => segments <= "1011111"; when X"7" => segments <= "1110000"; when X"8" => segments <= "1111111"; when X"9" => segments <= "1110011"; when others => segments <= "0000000"; end case; end process; end;</pre> 结束情况；结束 进程；结束； case语句检查data的值。当data为0时，该语句执行=>后的操作，将segments设置为1111110。case语句同样检查其他数据值，直至9（注意使用X表示十六进制数）。others子句是一种便捷方式，用于定义所有未明确列出的情况下的输出，从而确保组合逻辑的实现。 与SystemVerilog不同，VHDL支持选择信号赋值语句（见HDL示例4.6），该语句与case语句非常相似，但可以出现在进程外部。因此，使用进程描述组合逻辑的理由较少。
--	--

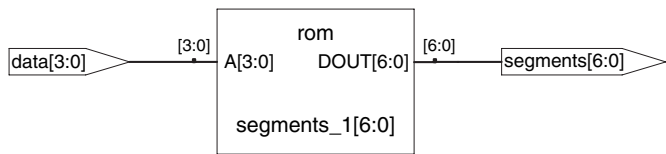


图4.20 七段数码管综合电路

定义了可能的输入组合。否则，它意味着时序逻辑，因为在未定义的情况下，输出将保持其旧值。

七段显示解码器的HDL综合成一个*read-only memory*（ROM），其中包含16种可能输入对应的7个输出。ROM将在第5.5.6节中进一步讨论。

如果case语句中省略了default或其他子句，当数据在10–15范围内时，解码器将记住其之前的输出。这对硬件来说是一种奇怪的行为。

普通解码器也通常使用case语句编写。HDL示例4.25描述了一个3:8解码器。

4.5.2 If 语句

always/process 语句也可以包含if语句。if语句后面可以跟一个else语句。如果所有可能的输入

HDL示例4.25 3:8译码器

SystemVerilog <pre>module decoder3_8(input logic [2:0] a, output logic [7:0] y); always_comb case(a) 3'b000: y=8'b00000001; 3'b001: y=8'b00000010; 3'b010: y=8'b00000100; 3'b011: y=8'b00001000; 3'b100: y=8'b00010000; 3'b101: y=8'b00100000; 3'b110: y=8'b01000000; 3'b111: y=8'b10000000; default: y=8'bxxxxxxxx; endcase endmodule</pre> The default statement isn't strictly necessary for logic synthesis in this case because all possible input combinations are defined, but it is prudent for simulation in case one of the inputs is an x or z.	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity decoder3_8 is port(a: in STD_LOGIC_VECTOR(2 downto 0); y: out STD_LOGIC_VECTOR(7 downto 0)); end; architecture synth of decoder3_8 is begin process(all) begin case a is when "000" => y <= "00000001"; when "001" => y <= "00000010"; when "010" => y <= "00000100"; when "011" => y <= "00001000"; when "100" => y <= "00010000"; when "101" => y <= "00100000"; when "110" => y <= "01000000"; when "111" => y <= "10000000"; when others => y <= "XXXXXXXX"; end case; end process; end;</pre> The others clause isn't strictly necessary for logic synthesis in this case because all possible input combinations are defined, but it is prudent for simulation in case one of the inputs is an x, z, or u.
--	--

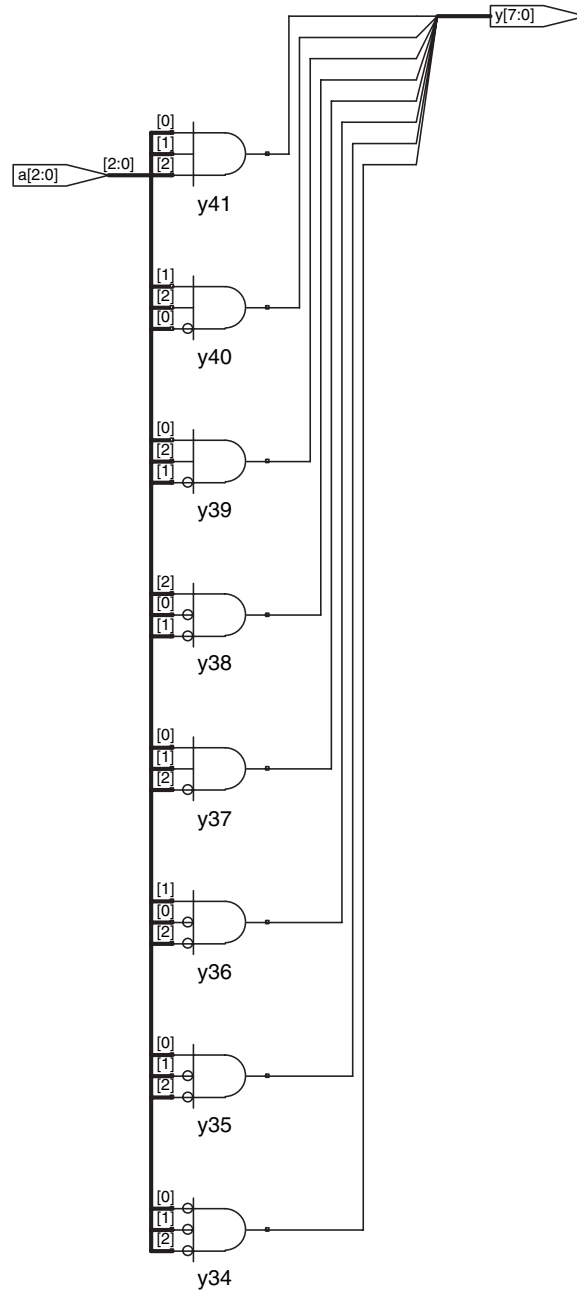


图4.21 decoder3_8综合电路

HDL示例4.26 优先级电路

SystemVerilog

```
module priorityckt(input logic [3:0] a,
                  output logic [3:0] y);
    always_comb
        if (a[3]) y = 4'b1000;
        else if (a[2]) y = 4'b0100;
        else if (a[1]) y = 4'b0010;
        else if (a[0]) y = 4'b0001;
        else
            y = 4'b0000;
endmodule
```

In SystemVerilog, if statements must appear inside of always statements.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity priorityckt is
    port(a: in STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of priorityckt is
begin
    process(all) begin
        if a(3) then y <= "1000";
        elsif a(2) then y <= "0100";
        elsif a(1) then y <= "0010";
        elsif a(0) then y <= "0001";
        else
            y <= "0000";
        end if;
    end process;
end;
```

Unlike SystemVerilog, VHDL supports conditional signal assignment statements (see HDL Example 4.6), which are much like if statements but can appear outside processes. Thus, there is less reason to use processes to describe combinational logic.

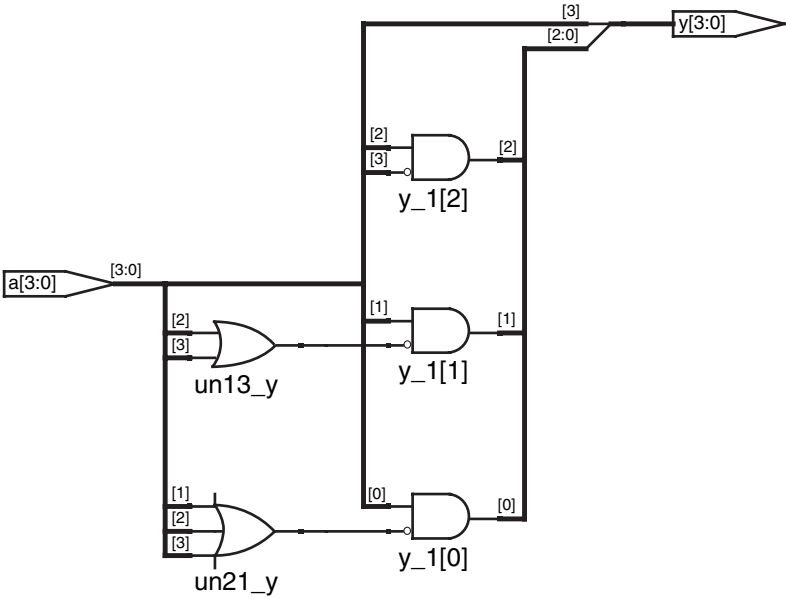


图4.22 priorityckt综合电路

组合逻辑被处理时，该语句意味着组合逻辑。否则，它会产生时序逻辑（如第4.4.5节中的锁存器）。

HDL示例4.26使用if语句描述了一个优先电路，该电路在第2.4节中定义。回顾一下，N输入优先电路会将输出设置为TRUE，该输出对应于最高位为TRUE的输入。

4.5.3 含无关项的真值表

如第2.7.3节所述，真值表可包含无关项以允许更多逻辑简化。HDL示例4.27展示了如何描述一个具有无关项的优先级电路。

综合工具为该模块生成的电路与图4.22中的优先级电路略有不同，如图4.23所示。然而，这些电路在逻辑上是等效的。

4.5.4 阻塞赋值与非阻塞赋值

第204页的指南说明了每种赋值类型的使用时机和方法。如果不遵循这些指南，可能会编写出在仿真中看似正确但综合后却生成错误硬件的代码。本节剩余的可选内容解释了这些指南背后的原理。

HDL示例4.27 使用无关项的优先级电路

SystemVerilog <pre>module priority_casez(input logic [3:0]a, output logic [3:0] y); always_comb casez(a) 4'b1???: y = 4'b1000; 4'b01??: y = 4'b0100; 4'b001?: y = 4'b0010; 4'b0001: y = 4'b0001; default: y = 4'b0000; endcase endmodule</pre> The casez statement acts like a case statement except that it also recognizes ? as don't care.	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity priority_casez is port(a: in STD_LOGIC_VECTOR(3 downto 0); y: out STD_LOGIC_VECTOR(3 downto 0)); end; architecture dontcare of priority_casez is begin process(all) begin case? a is when "1---" => y <= "1000"; when "01--" => y <= "0100"; when "001-" => y <= "0010"; when "0001" => y <= "0001"; when others => y <= "0000"; end case?; end process; end;</pre> The case? statement acts like a case statement except that it also recognizes - as don't care.
--	---

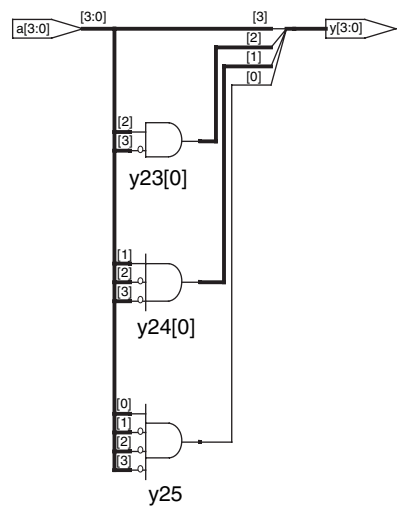


图4.23 priority_casez综合电路

阻塞与非阻塞赋值指南

SystemVerilog	VHDL
1. 使用always_ff @(posedge clk)和非阻塞赋值来建模同步时序逻辑。 <pre>always_ff @(posedge clk) begin n1 <= d; // 非阻塞 q <= n1; // 非阻塞 end</pre> 2. 使用连续赋值来建模简单的组合逻辑。 将 y = 赋值为 s ? d1 : d0; 3. 使用always_comb和阻塞赋值来建模更复杂的组合逻辑，此时always语句会很有帮助。 <pre>always_comb if (a[3]) y = 4'b1000; else if (a[2]) y = 4'b0100; else if (a[1]) y = 4' b0010; else if (a[0]) y = 4'b0001; else y = 4'b0000;</pre> 4. 不要在多个always语句或连续赋值语句中对同一信号进行赋值。	1. 使用process(clk)和非阻塞赋值来建模同步时序逻辑。 <pre>process(clk) begin if rising_edge(clk) then n1 <= d; -- 非阻塞 q <= n1; -- 非阻塞 end if; end process;</pre> 2. 在进程语句外部使用并发赋值来建模简单的组合逻辑。 当 s = 为 '0' 时，y <= 为 d0，否则为 d1; 3. 在进程有助于建模更复杂组合逻辑的情况下，使用 process(all) 进行建模。对内部变量使用阻塞赋值。 <pre>process(all) variable p, g; STD_LOGI C; begin p := a xor b; -- 阻塞赋值 g := a and b; -- 阻塞赋值 s <= p xor cin; cout <= g or (p and cin); end pro cess;</pre> 4. 不要在同一变量的多个进程或并发赋值语句中进行赋值。

组合逻辑*

HDL示例4.23中的全加器使用阻塞赋值正确建模。本节探讨其工作原理，以及若使用非阻塞赋值会产生的差异。

假设a、b和cin初始均为0。因此p、g、s和cout也均为0。在某个时刻，a变为1，触发always/process语句。四个阻塞赋值按此处所示的顺序执行。（在VHDL代码中，s和cout是并发赋值的。）请注意，由于阻塞赋值，p和g在计算s和cout之前就获得了新值。这一点很重要，因为我们希望使用p和g的新值来计算s和cout。

1. $p \leftarrow 1 \oplus 0 = 1$ 2. $g \leftarrow 1 \cdot 0 = 0$ 3. $s \leftarrow 1 \oplus 0 = 1$ 4. $\text{cout} \leftarrow 0 + 1 \cdot 0 = 0$

相比之下，HDL示例4.28展示了非阻塞赋值的使用。

现在，考虑相同的情况：当b和cin为0时，a从0上升到1。四个非阻塞赋值同时求值：

$p \leftarrow 1 \oplus 0 = 1$ $g \leftarrow 1 \cdot 0 = 0$ $s \leftarrow 0 \oplus 0 = 0$ $\text{cout} \leftarrow 0 + 0 \cdot 0 = 0$

HDL示例4.28 使用非阻塞赋值的全加器

SystemVerilog	VHDL
<pre>// nonblocking assignments (not recommended) module fulladder(input logic a, b, cin, output logic s, cout); logic p, g; always_comb begin p <= a ^ b; // nonblocking g <= a & b; // nonblocking s <= p ^ cin; cout <= g (p & cin); end endmodule</pre>	<pre>-- nonblocking assignments (not recommended) library IEEE; use IEEE.STD_LOGIC_1164.all; entity fulladder is port(a, b, cin: in STD_LOGIC; s, cout: out STD_LOGIC); end; architecture nonblocking of fulladder is signal p, g: STD_LOGIC; begin process(all) begin p <= a xor b; -- nonblocking g <= a and b; -- nonblocking s <= p xor cin; cout <= g or (p and cin); end process; end;</pre> Because p and g appear on the left-hand side of a nonblocking assignment in a process statement, they must be declared to be signal rather than variable. The signal declaration appears before the begin in the architecture, not the process.

注意，s与p是同时计算的。因此，它使用的是p的旧值，而非新值。所以，s保持为0而不是变为1。然而，p确实从0变为1。这一变化触发always/process语句再次求值，如下所示：

$$p \leftarrow 1 \oplus 0 = 1 \quad g \leftarrow 1 \cdot 0 = 0 \quad s \leftarrow 1 \oplus 0 = 1 \quad \text{cout} \leftarrow 0 + 1 \cdot 0 = 0$$

这次，p已经是1，所以s正确变为1。非阻塞赋值最终得到了正确结果，但always/process语句不得不执行两次。虽然综合出的硬件相同，但这使得仿真速度变慢。

在建模组合逻辑时，非阻塞赋值的另一个缺点是，如果忘记将中间变量包含在敏感列表中，HDL将产生错误结果。

更糟糕的是，即使错误的敏感列表导致仿真不正确，某些综合工具仍会综合出正确的硬件。这会导致仿真结果与实际硬件行为不匹配。

SystemVerilog	VHDL
如果HDL示例4.28中always语句的敏感列表写成always @(a, b, cin)而非always_comb，那么当p发生变化时，该语句将不会重新求值。 或者 g 发生变化。在这种情况下，s 会被错误地保留为 0，而不是 1。	如果HDL示例4.28中过程语句的敏感列表写成process(a, b, cin)而不是process(all)，那么当p或g发生变化时，该语句将不会重新求值。在这种情况下，s将被错误地保持在0，而不是1。

时序逻辑*

HDL示例4.20中的同步器使用非阻塞赋值正确建模。在时钟上升沿，d被复制到n1的同时，n1被复制到q，因此代码正确描述了两个寄存器。例如，假设初始状态为d = 0、n1 = 1、q = 0。在时钟上升沿，以下两个赋值同时发生，因此在时钟边沿后，n1 = 0且q = 1。

$$n1 \leftarrow d = 0 \quad q \leftarrow n1 = 1$$

HDL示例4.29尝试使用阻塞赋值来描述同一模块。在clk的上升沿，d被复制到n1。然后，n1的这个新值被复制到q，导致d错误地同时出现在n1和q。赋值依次发生，因此在时钟边沿之后，q = n1 = 0。

1. $n1 \leftarrow d = 0$
2. $q \leftarrow n1 = 0$

HDL示例4.29 使用阻塞赋值的不良同步器

SystemVerilog <pre>// Bad implementation of a synchronizer using blocking // assignments module syncbad(input logic clk, input logic d, output logic q); logic n1; always_ff @(posedge clk) begin n1 = d; // blocking q = n1; // blocking end endmodule</pre>	VHDL <pre>-- Bad implementation of a synchronizer using blocking -- assignment library IEEE; use IEEE.STD_LOGIC_1164.all; entity syncbad is port(clk:in STD_LOGIC; d: in STD_LOGIC; q: out STD_LOGIC); end; architecture bad of syncbad is begin process(clk) variable n1: STD_LOGIC; begin if rising_edge(clk) then n1 := d; -- blocking q <= n1; end if; end process; end;</pre>
--	--

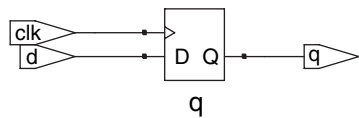


图4.24 syncbad综合电路

由于n1对外部世界不可见，且不影响q的行为，综合器将其完全优化掉，如图4.24所示。

此示例的寓意在于，当对时序逻辑进行建模时，应在always/process语句中专门使用非阻塞赋值。即便足够巧妙，例如通过调整赋值顺序，也能使阻塞赋值正确工作，但阻塞赋值并无优势，只会引入意外行为的风险。某些时序电路无论采用何种顺序，都无法通过阻塞赋值正常工作。

4.6 有限状态机

回顾有限状态机（FSM）由状态寄存器和两个组合逻辑块组成，用于根据当前状态和输入计算下一状态及输出，如图3.22所示。状态机的HDL描述相应地分为三个部分，分别对状态寄存器、下一状态逻辑和输出逻辑进行建模。

HDL示例4.30 除以3的有限状态机

SystemVerilog

```
module divideby3FSM(input logic clk,
                   input logic reset,
                   output logic y);
    typedef enum logic [1:0] {S0, S1, S2} statetype;
    statetype state, nextstate;

    // state register
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    // next state logic
    always_comb
        case (state)
            S0:      nextstate = S1;
            S1:      nextstate = S2;
            S2:      nextstate = S0;
            default: nextstate = S0;
        endcase

    // output logic
    assign y = (state == S0);
endmodule
```

The `typedef` statement defines `statetype` to be a two-bit 具有三种可能性的逻辑值：S0、S1或S2。state和nextstate是statetype信号。

枚举编码默认按数值顺序排列：S0 = 00，S1 = 01，S2 = 10。用户可显式设置编码；然而，综合工具将其视为建议而非强制要求。例如，以下代码片段将状态编码为3位独热值：

```
typedef enum logic [2:0] {S0 = 3'b001, S1 = 3'b010, S2 = 3'b100} statetype;
```

注意case语句是如何用于定义状态转换表的。由于下一状态逻辑应为组合逻辑，因此即使状态2'b11不应出现，也必须包含默认项。

输出y在状态为S0时为1。equality comparison `a == b`在a等于b时结果为1，否则为0。inequality comparison `a != b`执行相反操作，在a不等于b时结果为1。

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity divideby3FSM is
    port(clk, reset: in STD_LOGIC;
         y: out STD_LOGIC);
end;

architecture synth of divideby3FSM is
    type statetype is (S0, S1, S2);
    signal state, nextstate: statetype;
begin

    -- state register
    process(clk, reset) begin
        if reset then state <= S0;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    -- next state logic
    nextstate <= S1 when state = S0 else
                S2 when state = S1 else
                S0;

    -- output logic
    当状态=为S0时, y <= '1', 否则为'0'; 结束
;
```

本示例定义了一个新的enumeration数据类型statetype，包含三种可能状态：S0、S1和S2。state和nextstate是statetype类型的信号。通过使用枚举而非选择状态编码，VHDL使综合器能够自由探索多种状态编码，从而选择最优方案。

在上述HDL中，当状态为S0时，输出y为1。不等式比较使用/=。若要在状态不为S0时输出1，请将比较改为state /= S0。

HDL 示例 4.30 描述了第 3.4.2 节中的三分频有限状态机。它提供了一个异步复位来初始化该有限状态机。状态寄存器使用了触发器的常规写法。下一状态和输出逻辑块是组合逻辑。

综合工具仅生成状态机的框图和状态转换图；它们不显示逻辑门或输入。

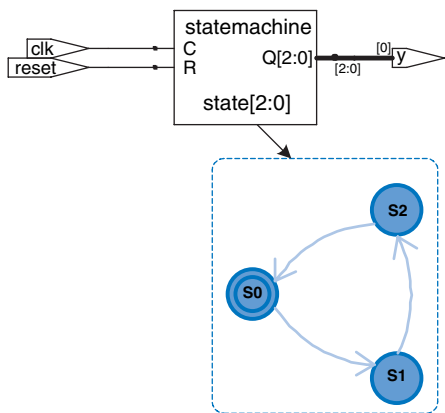


图4.25 divideby3fsm 综合电路

在弧线和状态上都有输出。因此，请务必在HDL代码中正确指定FSM。图4.25中三分频FSM的状态转换图与图3.28(b)中的图类似。双圆圈表示S0是复位状态。三分频FSM的门级实现在第3.4.2节中已展示。

请注意，综合工具使用了3位编码(Q[2:0])，而非SystemVerilog代码中建议的2位编码。

注意，状态使用枚举数据类型命名，而非以二进制值引用。这使得代码更易读且更易于修改。

如果出于某种原因，我们希望输出在状态S0和S1时为高电平，则输出逻辑将修改如下。

SystemVerilog	VHDL
<pre>// output logic assign y = (state == S0 state == S1);</pre>	<pre>-- output logic y <= '1' when (state = S0 or state = S1) else '0';</pre>

接下来的两个示例描述了第3.4.3节中的蜗牛模式识别器FSM。代码展示了如何使用case和if语句来处理依赖于输入以及当前状态的下一状态和输出逻辑。我们同时展示了Moore和Mealy模块。在Moore机（HDL示例4.31）中，输出仅取决于当前状态，而在Mealy机（HDL示例4.32）中，输出逻辑同时取决于当前状态和输入。

HDL示例4.31 模式识别器摩尔型有限状态机

SystemVerilog

```
module patternMoore(input  logic clk,
                    input  logic reset,
                    input  logic a,
                    output logic y);

    typedef enum logic [1:0] {S0, S1, S2} statetype;
    statetype state, nextstate;

    // state register
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    // next state logic
    always_comb
        case (state)
            S0: if (a) nextstate = S0;
                else nextstate = S1;
            S1: if (a) nextstate = S2;
                else nextstate = S1;
            S2: if (a) nextstate = S0;
                else nextstate = S1;
            default: nextstate = S0;
        endcase

    // output logic
    assign y = (state == S2);
endmodule
```

Note how nonblocking assignments (<=) are used in the state register to describe sequential logic, whereas blocking assignments (=) are used in the next state logic to describe combinational logic.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity patternMoore is
    port(clk, reset: in  STD_LOGIC;
         a:      in  STD_LOGIC;
         y:      out STD_LOGIC);
end;

architecture synth of patternMoore is
    type statetype is (S0, S1, S2);
    signal state, nextstate: statetype;
begin
    -- state register
    process(clk, reset) begin
        if reset then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    -- next state logic
    process(all) begin
        case state is
            when S0 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when S1 =>
                if a then nextstate <= S2;
                else      nextstate <= S1;
                end if;
            when S2 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when others =>
                nextstate <= S0;
        end case;
    end process;

    --output logic
    y <= '1' when state = S2 else '0';
end;
```

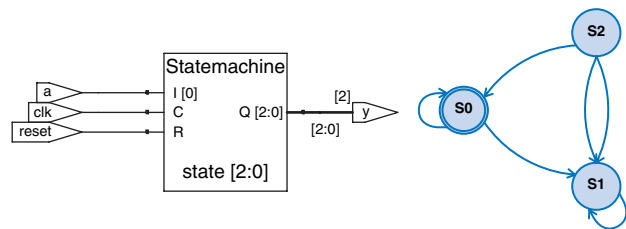


图4.26 Moore型模式合成电路

HDL示例4.32 模式识别器米利型有限状态机

SystemVerilog <pre>module patternMealy(input logic clk, input logic reset, input logic a, output logic y); typedef enum logic {S0, S1} statetype; statetype state, nextstate; // state register always_ff @(posedge clk, posedge reset) if (reset) state <= S0; else state <= nextstate; // next state logic always_comb case (state) S0: if (a) nextstate = S0; else nextstate = S1; S1: if (a) nextstate = S0; else nextstate = S1; default: nextstate = S0; endcase // output logic assign y = (a & state == S1); endmodule</pre>	VHDL <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity patternMealy is port(clk, reset: in STD_LOGIC; a: in STD_LOGIC; y: out STD_LOGIC); end; architecture synth of patternMealy is type statetype is (S0, S1); signal state, nextstate: statetype; begin -- state register process(clk, reset) begin if reset then state <= S0; elsif rising_edge(clk) then state <= nextstate; end if; end process; -- next state logic process(all) begin case state is when S0 => if a then nextstate <= S0; else nextstate <= S1; end if; when S1 => if a then nextstate <= S0; else nextstate <= S1; end if; when others => nextstate <= S0; end case; end process; -- output logic y <= '1' when (a = '1' and state = S1) else '0'; end;</pre>
--	---

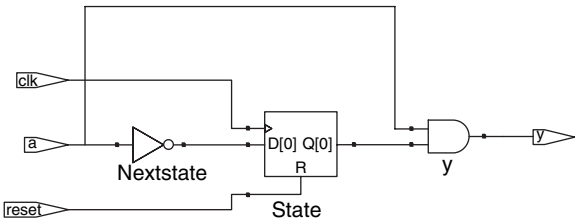


图4.27 patternMealy综合电路

4.7 数据类型*

本节更深入地解释了SystemVerilog和VHDL类型中的一些微妙之处。

4.7.1 SystemVerilog

在SystemVerilog之前，Verilog主要使用两种类型：reg和wire。尽管名为reg，但reg信号可能与寄存器相关，也可能不相关。这给学习该语言的人带来了极大的困惑。SystemVerilog引入了logic类型以消除这种混淆，因此本书重点强调logic类型。本节将更详细地解释reg和wire类型，供需要阅读旧版Verilog代码的读者参考。

在Verilog中，如果某个信号出现在always块中 $\leq$ 或 $=$ 的左侧，则必须声明为reg。否则，应声明为wire。因此，reg信号可能是触发器、锁存器或组合逻辑的输出，具体取决于always块的敏感列表和语句。

输入和输出端口默认类型为wire，除非其类型被显式定义为reg。以下示例展示了如何在传统Verilog中描述触发器。请注意，clk和d默认为wire类型，而q被显式定义为reg，因为它出现在always块中 $\leq$ 的左侧。

```
模块 flop(输入      clk,      输入  [3:0]
d,      输出 reg [3:0] q); 始终 @(posedge cl
k)  q <= d; 结束模块
```

SystemVerilog 引入了 logic 类型。logic 是 reg 的同义词，避免了误导用户认为它一定是触发器。此外，SystemVerilog 放宽了对 assign 语句和层次化端口实例化的规则，使得 logic 可以在传统上需要 wire 的 always 块外部使用。因此，几乎所有 SystemVerilog 信号都可以是 logic。例外情况是，具有多个驱动器的信号（例如三态总线）必须声明为 net，如 HDL 示例 4.10 所述。这一规则使得 SystemVerilog 能够在 logic 信号意外连接到多个驱动器时生成错误消息，而不是产生 x 值。

最常见的网络类型称为wire或tri。这两种类型是同义词，但通常当存在单个驱动源时使用wire，而当存在多个驱动源时使用tri。因此，在SystemVerilog中wire已过时，因为对于具有单个驱动源的信号，更推荐使用logic。

当三态网络被一个或多个驱动源驱动至单一值时，其呈现该值。当未被驱动时，其处于高阻态（z）。当被多个驱动源驱动至不同值（0、1或x）时，则处于竞争状态（x）。

还有其他网络类型在未驱动或多源驱动时解析方式不同。这些类型很少使用，但可以替代任何通常出现三态网络的位置（例如，用于具有多个驱动器的信号）。每种类型在表4.7中描述。

表4.7 净分辨率

Net Type	No Driver	Conflicting Drivers
tri	z	x
triereg	previous value	x
triand	z	0 if there are any 0
trior	z	1 if there are any 1
tri0	0	x
tri1	1	x

4.7.2 VHDL

与SystemVerilog不同，VHDL强制执行严格的数据类型系统，这可以保护用户免受某些错误的影响，但有时也显得笨拙。

尽管STD_LOGIC类型具有根本重要性，但它并非VHDL的内置类型，而是IEEE.STD_LOGIC_1164库的一部分。因此，每个文件都必须包含前述示例中所示的库声明语句。

此外，IEEE.STD_LOGIC_1164 缺少针对 STD_LOGIC_VECTOR 数据的基本操作，如加法、比较、移位以及转换为整数。这些操作最终在 VHDL 2008 标准的 IEEE.NUMERIC_STD_UNSIGNED 库中被添加。

VHDL 还有一种 BOOLEAN 类型，包含两个值：true 和 false。BOOLEAN 值由比较操作（如相等比较 `s='0'`）返回，并用于条件语句中，例如 `when` 和 `if`。尽管人们可能倾向于认为 BOOLEAN 的 true 值应等同于 STD_LOGIC 的 '1'，而 BOOLEAN 的 false 值应等同于 STD_LOGIC 的 '0'，但在 VHDL 2008 之前，这些类型是不可互换的。例如，在旧的 VHDL 代码中，必须这样写：

```
y <= d1 when (s='1') else d0;
```

而在VHDL 2008中，`when`语句会自动将s从STD_LOGIC转换为BOOLEAN，因此可以直接编写

当s为真时，`y <= d1`，否则为d0；

即使在VHDL 2008中，仍然需要编写

```
q <= '1' when (state=S2) else '0';
```

而不是

```
q <= (state=S2);
```

因为(state = S2)返回一个BOOLEAN结果, 不能直接赋值给STD_LOGIC信号y。

尽管我们未将任何信号声明为布尔类型, 但比较操作会自动隐含布尔属性, 且条件语句会使用该属性。类似地, VHDL 具有表示正负整数的 INTEGER 类型。INTEGER 类型的信号取值范围至少为 $-(2^{31} - 1)$ 到 $2^{31} - 1$ 。整数值用作总线索引。例如, 在以下语句中

```
y <= a(3) 和 a(2) 和 a(1) 和 a(0);
```

0、1、2和3是整数, 用作选择a信号比特位的索引。我们不能直接用STD_LOGIC或STD_LOGIC_VECTOR信号对总线进行索引, 而必须将该信号转换为INTEGER。下面的示例演示了这一点, 该示例是一个8:1多路选择器, 使用3位索引从向量中选择一个比特位。TO_INTEGER函数定义在IEEE.NUMERIC_STD_UNSIGNED库中, 用于将STD_LOGIC_VECTOR转换为INTEGER, 适用于正(无符号)值。

```
库 IEEE; 使用 IEEE.STD_LOGIC_1164.所有; 使用 IEEE.
NUMERIC_STD_UNSIGNED.所有; 实体 mux8 是 端口(d:
输入 STD_LOGIC_VECTOR(7 downto 0);      s: 输入 ST
D_LOGIC_VECTOR(2 downto 0);      y: 输出 STD_LOGI
C); 结束; 架构 synth of mux8 是开始 y <= d(TO_INTE
GER(s)); 结束;
```

VHDL 对输出端口仅用于输出也有严格规定。例如, 以下用于二输入和三输入与门的代码在 VHDL 中是非法的, 因为 v 是输出端口, 同时也被用于计算 w。

```
库 IEEE; 使用 IEEE.STD_LOGIC_1164.全部;
实体 and23 端口(a, b, c: 输入 STD_LOGI
C; v, w: 输出 STD_LOGIC); 结束;
```

```
architecture synth of and23 is begin v <= a
and b; w <= v and c; end;
```

VHDL定义了一种特殊的端口类型buffer来解决这个问题。连接到buffer端口的信号表现为输出, 但也可以在模块内部使用。修正后的实体定义如下。Verilog

而 SystemVerilog 没有此限制，也不需要缓冲端口。VHDL 2008 通过允许输出端口可读来消除这一限制。

实体 and23 是 端口(a, b, c: 输入 STD_LOGIC;
v: 缓冲 STD_LOGIC; w:
输出 STD_LOGIC); 结束

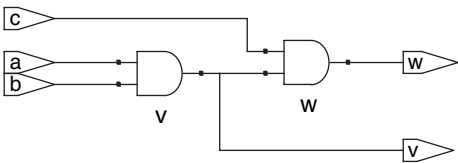


图4.28 和23合成的电路

大多数运算（如加法、减法和布尔逻辑）无论数字是有符号还是无符号，其操作方式都是相同的。然而，对于有符号二进制补码数和无符号二进制数，大小比较、乘法和算术右移的执行方式有所不同。这些运算将在第 5 章中讨论。HDL 示例 4.33 描述了如何指示信号表示有符号数。

HDL 示例 4.33 (a) 无符号乘法器 (b) 有符号乘法器

SystemVerilog // 4.33(a): unsigned multiplier module multiplier(input logic [3:0] a, b, output logic [7:0] y); assign y = a * b; endmodule // 4.33(b): signed multiplier module multiplier(input logic signed [3:0] a, b, output logic signed [7:0] y); assign y = a * b; endmodule In SystemVerilog, signals are considered unsigned by default. Adding the signed modifier (e.g., logic signed [3:0] a) causes the signal a to be treated as a signed—that is, two’s complement—number. VHDL -- 4.33(a): unsigned multiplier library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all; entity multiplier is port(a, b: in STD_LOGIC_VECTOR(3 downto 0); y: out STD_LOGIC_VECTOR(7 downto 0)); end; architecture synth of multiplier is begin y <= a * b; end; VHDL uses the NUMERIC_STD_UNSIGNED library to perform arithmetic and comparison operations on STD_LOGIC_VECTORS. The vectors are treated as unsigned. use IEEE.NUMERIC_STD_UNSIGNED.all; VHDL also defines UNSIGNED and SIGNED data types in the IEEE.NUMERIC_STD library, but these involve type conversions beyond the scope of this chapter.
--

4.8 参数化模块*

到目前为止，我们所有的模块都具有固定宽度的输入和输出。例如，我们必须为 4 位和 8 位宽的 2:1 多路复用器分别定义模块。HDL 允许使用参数化模块实现可变位宽。

HDL 示例 4.34 声明了一个默认宽度为 8 的参数化 2:1 多路选择器，然后使用它创建了 8 位和 12 位的 4:1 多路选择器。

HDL示例4.34 参数化N位2:1多路选择器

SystemVerilog 模块 mux2 #(参数 width = 8) (输入 逻辑 [width-1:0] d0, d1, 输入 逻辑 s, 输出 逻辑 [width-1:0] y); 赋值 y = s ? d1 : d0; 结束模块 SystemVerilog 允许在输入和输出之前使用 #(parameter ...) 语句来定义参数。该参数语句包含参数的默认值 (8), 在此例中参数名为 width。输入和输出的位数可以依赖于该参数。 模块 mux4_8(输入 逻辑 [7:0] d0, d1, d2, d3, 输入 逻辑 [1:0] s, 输出 逻辑 [7:0] y); 逻辑 [7:0] low, hi; mux2 lowmux(d0, d1, s[0], low); mux2 himux(d2, d3, s[0], hi); mux2 outmux(low, hi, s[1], y); endmodule 8位4:1多路选择器使用默认宽度实例化了三个2:1多路选择器。 相比之下, 一个12位4:1多路选择器mux4_12需要在实例名称前使用#()覆盖默认宽度, 如下所示。 模块 mux4_12(输入 逻辑 [11:0] d0, d1, d2, d3, 输入 逻辑 [1:0] s, 输出 逻辑 [11:0] y); 逻辑 [11:0] low, hi; mux2 #(12) lowmux(d0, d1, s[0], low); mux2 #(12) himux(d2, d3, s[0], hi); mux2 #(12) outmux(low, hi, s[1], y); endmodule 不要将表示延迟的#符号与在定义和覆盖参数时使用的#(...)混淆。 <td> VHDL 库 IEEE; 使用 IEEE.STD_LOGIC_1164.所有; 实体 mux2 是 泛型(宽度: 整数 := 8); 端口(d0, d1: 输入 STD_LOGIC_VECTOR(宽度-1 downto 0); s: 输入 STD_LOGIC; y: 输出 STD_LOGIC_VECTOR(宽度-1 downto 0)); 结束; 架构 synt h of mux2 是 开始 y <= d1 当 s 否则 d0; 结束; 通用语句包含一个默认宽度值 (8)。该值为整数。 库 IEEE; 使用 IEEE.STD_LOGIC_1164.全部; 实体 mux4_8 是 端口(d0, d1, d2, d3: 输入 STD_LOGIC_VECTOR(7 downto 0); s: 输入 STD_LOGIC_VECTOR(1 downto 0); y: 输出 STD_LOGIC_VECTOR(7 downto 0)); 结束; mux4_8的架构结构为 组件 mux2 泛型(宽度: 整数 := 8); 端口(d0, d1: 输入 STD_LOGIC_VECTOR(宽度-1 downto 0); s: 输入 STD_LOGIC; y: 输出 STD_LOGIC_VECTOR(宽度-1 downto 0)); 结束组件; 信号 low, hi: STD_LOGIC_VECTOR(7 downto 0); 开始lowmux: mux2 端口映射(d0, d1, s(0), low); himux: mux2 端口映射(d2, d3, s(0), hi); outmux: mux2 端口映射(low, hi, s(1), y); 结束 8位4:1多路选择器mux4_8实例化了三个2:1多路选择器, 并使用它们的默认宽度。 相比之下, 一个12位4:1多路选择器mux4_12需要通过通用映射 (generic map) 覆盖默认宽度, 如下所示。 lowmux: mux2 generic map(12) port map(d0, d1, s(0), low); himux: mux2 generic map(12) port map(d2, d3, s(0), hi); outmux: mux2 generic map(12) port map(low, hi, s(1), y); </td>	VHDL 库 IEEE; 使用 IEEE.STD_LOGIC_1164.所有; 实体 mux2 是 泛型(宽度: 整数 := 8); 端口(d0, d1: 输入 STD_LOGIC_VECTOR(宽度-1 downto 0); s: 输入 STD_LOGIC; y: 输出 STD_LOGIC_VECTOR(宽度-1 downto 0)); 结束; 架构 synt h of mux2 是 开始 y <= d1 当 s 否则 d0; 结束; 通用语句包含一个默认宽度值 (8)。该值为整数。 库 IEEE; 使用 IEEE.STD_LOGIC_1164.全部; 实体 mux4_8 是 端口(d0, d1, d2, d3: 输入 STD_LOGIC_VECTOR(7 downto 0); s: 输入 STD_LOGIC_VECTOR(1 downto 0); y: 输出 STD_LOGIC_VECTOR(7 downto 0)); 结束; mux4_8的架构结构为 组件 mux2 泛型(宽度: 整数 := 8); 端口(d0, d1: 输入 STD_LOGIC_VECTOR(宽度-1 downto 0); s: 输入 STD_LOGIC; y: 输出 STD_LOGIC_VECTOR(宽度-1 downto 0)); 结束组件; 信号 low, hi: STD_LOGIC_VECTOR(7 downto 0); 开始lowmux: mux2 端口映射(d0, d1, s(0), low); himux: mux2 端口映射(d2, d3, s(0), hi); outmux: mux2 端口映射(low, hi, s(1), y); 结束 8位4:1多路选择器mux4_8实例化了三个2:1多路选择器, 并使用它们的默认宽度。 相比之下, 一个12位4:1多路选择器mux4_12需要通过通用映射 (generic map) 覆盖默认宽度, 如下所示。 lowmux: mux2 generic map(12) port map(d0, d1, s(0), low); himux: mux2 generic map(12) port map(d2, d3, s(0), hi); outmux: mux2 generic map(12) port map(low, hi, s(1), y);
---	--

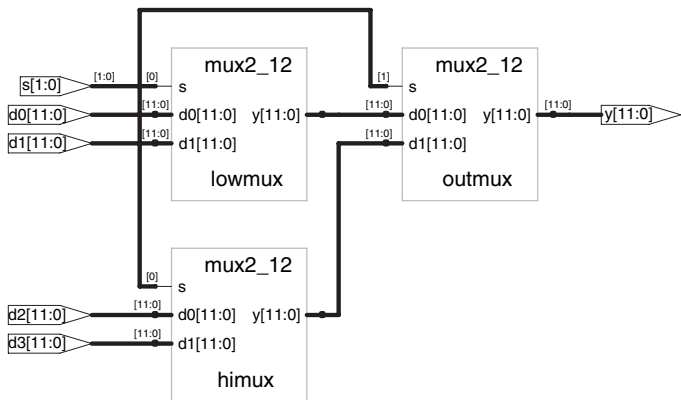


图4.29 mux4_12综合电路

HDL示例4.35 参数化 $N:2^N$ 解码器

SystemVerilog	VHDL
<pre>module decoder #(parameter N = 3) (input logic [N-1:0] a, output logic [2**N-1:0] y); always_comb begin y = 0; y[a] = 1; end endmodule</pre> 2^{**N} indicates 2^N.	<pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD.UNSIGNED.all; entity decoder is generic(N: integer := 3); port(a: in STD_LOGIC_VECTOR(N-1 downto 0); y: out STD_LOGIC_VECTOR(2**N-1 downto 0)); end; architecture synth of decoder is begin process(all) begin y <= (OTHERS => '0'); y(TO_INTEGER(a)) <= '1'; end process; end;</pre> 2^{**N} indicates 2^N.

HDL示例4.35展示了一个解码器，这是参数化模块的一个更佳应用。使用case语句来指定一个大型的 $N:2^N$ 解码器非常繁琐，但通过参数化代码却很容易实现，只需将相应的输出位设置为1即可。具体来说，该解码器使用阻塞赋值将所有位设置为0，然后将相应的位改为1。

HDL也提供生成语句，可根据参数值产生可变数量的硬件。生成语句支持for循环和if语句，以确定要生成多少种及何种类型的硬件。HDL示例4.36演示了如何使用生成语句从N输入与门函数中生成。

两级两输入与门的级联。当然，对于此应用，归约运算符会更简洁、更简单，但该示例说明了硬件生成器的一般原理。

谨慎使用generate语句；它很容易无意中产生大量硬件！

HDL示例4.36 参数化N输入与门

SystemVerilog

```
module andN
#(parameter width = 8)
(input logic [width-1:0] a,
 output logic y);

genvar i;
logic [width-1:0] x;

generate
  assign x[0] = a[0];
  for(i=1; i<width; i=i+1) begin: forloop
    assign x[i] = a[i] & x[i-1];
  end
endgenerate

assign y = x[width-1];
endmodule
```

The for statement loops through $i = 1, 2, \dots, \text{width}-1$ to produce many consecutive AND gates. The begin in a generate for loop must be followed by a : and an arbitrary label (forloop, in this case).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity andN is
  generic(width: integer := 8);
  port(a: in STD_LOGIC_VECTOR(width-1 downto 0);
        y: out STD_LOGIC);
end;

architecture synth of andN is
  signal x: STD_LOGIC_VECTOR(width-1 downto 0);
begin
  x(0) <= a(0);
  gen: for i in 1 to width-1 generate
    x(i) <= a(i) and x(i-1);
  end generate;
  y <= x(width-1);
end;
```

The generate loop variable i does not need to be declared.

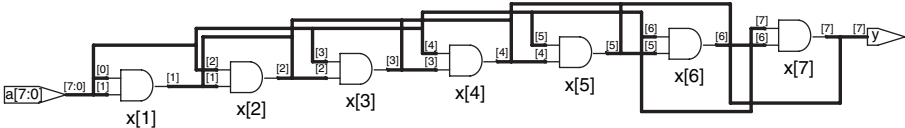


图4.30 与N合成电路

4.9 测试平台

一些工具也将待测试的模块称为unit（UUT）。

testbench是一种HDL模块，用于测试另一个称为device under test（DUT）的模块。测试平台包含向待测设计（DUT）施加输入的语句，理想情况下还包含检查是否产生正确输出的语句。输入和期望输出模式称为test vectors。

考虑测试第4.1.1节中的sillyfunction模块，该模块计算 $y = a b c a b c a b c$ 。这是一个简单的模块，因此我们可以通过应用所有八个可能的测试向量来进行穷举测试。

HDL示例4.37 测试平台

SystemVerilog

```

module testbench1();
    logic a, b, c, y;

    // instantiate device under test
    sillyfunction dut(a, b, c, y);

    // apply inputs one at a time
    initial begin
        a = 0; b = 0; c = 0; #10;
        c = 1;          #10;
        b = 1; c = 0;    #10;
        c = 1;          #10;
        a = 1; b = 0; c = 0; #10;
        c = 1;          #10;
        b = 1; c = 0;    #10;
        c = 1;          #10;
    end
endmodule

```

The `initial` statement executes the statements in its body at the start of simulation. In this case, it first applies the input pattern 000 and waits for 10 time units. It then applies 001 and waits 10 more units, and so forth until all eight possible inputs have been applied. `initial` statements should be used only in testbenches for simulation, not in modules intended to be synthesized into actual hardware. Hardware has no way of magically executing a sequence of special steps when it is first turned on.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench1 is -- no inputs or outputs
end;

architecture sim of testbench1 is
    component sillyfunction
        port(a, b, c: in  STD_LOGIC;
              y:      out STD_LOGIC);
    end component;
    signal a, b, c, y: STD_LOGIC;
begin
    -- instantiate device under test
    dut: sillyfunction port map(a, b, c, y);

    -- apply inputs one at a time
    process begin
        a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
        c <= '1';          wait for 10 ns;
        b <= '1'; c <= '0';    wait for 10 ns;
        c <= '1';          wait for 10 ns;
        a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
        c <= '1';          wait for 10 ns;
        b <= '1'; c <= '0';    wait for 10 ns;
        c <= '1';          wait for 10 ns;
        wait; --wait forever
    end process;
end;

```

The `process` statement first applies the input pattern 000 and waits for 10 ns. It then applies 001 and waits 10 more ns, and so forth until all eight possible inputs have been applied.

At the end, the process waits indefinitely. Otherwise, the process would begin again, repeatedly applying the pattern of test vectors.

HDL示例4.37展示了一个简单的测试平台。它实例化待测设计（DUT），然后施加输入。使用阻塞赋值和延迟来按适当顺序施加输入。用户必须查看仿真结果并通过检查验证是否产生了正确的输出。测试平台的仿真方式与其他HDL模块相同，但不可综合。

检查正确输出既繁琐又容易出错。此外，在设计思路清晰时确定正确输出要容易得多。如果几周后你做了微小改动并需要重新测试，确定正确输出就会变得很麻烦。更好的方法是编写一个自测试平台，如HDL示例4.38所示。

为每个测试向量编写代码也变得繁琐，尤其是对于需要大量向量的模块。一种更好的

HDL示例4.38 自检测试平台

SystemVerilog

```
module testbench2();
  logic a, b, c, y;

  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    assert (y === 1) else $error("000 failed.");
    c = 1; #10;
    assert (y === 0) else $error("001 failed.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("010 failed.");
    c = 1; #10;
    assert (y === 0) else $error("011 failed.");
    a = 1; b = 0; c = 0; #10;
    assert (y === 1) else $error("100 failed.");
    c = 1; #10;
    assert (y === 1) else $error("101 failed.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("110 failed.");
    c = 1; #10;
    assert (y === 0) else $error("111 failed.");
  end
endmodule
```

The SystemVerilog `assert` statement checks whether a specified condition is true. If not, it executes the `else` statement. The `$error` system task in the `else` statement prints an error message describing the assertion failure. `assert` is ignored during synthesis.

In SystemVerilog, comparison using `==` or `!=` is effective between signals that do not take on the values of `x` and `z`. Testbenches use the `===` and `!==` operators for comparisons of equality and inequality, respectively, because these operators work correctly with operands that could be `x` or `z`.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench2 is -- no inputs or outputs
end;

architecture sim of testbench2 is
  component sillyfunction
    port(a, b, c: in STD_LOGIC;
         y: out STD_LOGIC);
  end component;
  signal a, b, c, y: STD_LOGIC;
begin
  -- instantiate device under test
  dut: sillyfunction port map(a, b, c, y);

  -- apply inputs one at a time
  -- checking results
  process begin
    a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "000 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "001 failed.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "010 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "011 failed.";
    a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "100 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '1' report "101 failed.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "110 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "111 failed.";
    wait; -- wait forever
  end process;
end;
```

The `assert` statement checks a condition and prints the message given in the `report` clause if the condition is not satisfied. `assert` is meaningful only in simulation, not in synthesis.

方法是将测试向量放在一个单独的文件中。测试平台只需从文件中读取测试向量，将输入测试向量应用于待测设计（DUT），等待，检查DUT的输出值是否与输出向量匹配，并重复此过程，直到到达测试向量文件的末尾。

HDL 示例 4.39 展示了这样一个测试平台。该测试平台使用没有敏感列表的 `always/process` 语句生成时钟，从而使其持续被重新评估。在仿真开始时，它从文本文件中读取测试向量，并将复位信号脉冲两个周期。尽管时钟和复位对于测试组合逻辑并非必需，但它们被包含在内，因为当

测试一个时序DUT。example.txt是一个包含*test vectors*的文本文件，输入和预期输出以二进制形式写入：

```
000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0
```

HDL示例4.39 带测试向量文件的测试平台

SystemVerilog

```
module testbench3();
    logic      clk, reset;
    logic      a, b, c, y, yexpected;
    logic [31:0] vectornum, errors;
    logic [3:0] testvectors[10000:0];

    // instantiate device under test
    sillyfunction dut(a, b, c, y);

    // generate clock
    always
    begin
        clk = 1; #5; clk = 0; #5;
    end

    // at start of test, load vectors
    // and pulse reset
    initial
    begin
        $readmemb("example.txt", testvectors);
        vectornum = 0; errors = 0;
        reset = 1; #22; reset = 0;
    end

    // apply test vectors on rising edge of clk
    always @(posedge clk)
    begin
        #1; {a, b, c, yexpected} = testvectors[vectornum];
    end

    // check results on falling edge of clk
    always @(negedge clk)
    if (~reset) begin // skip during reset
        if (y !== yexpected) begin // check result
            $display("Error: inputs = %b", {a, b, c});
            $display(" outputs = %b (%b expected)", y, yexpected);
            errors = errors + 1;
        end
        vectornum = vectornum + 1;
        if (testvectors[vectornum] === 4'bx) begin
            $display("%d tests completed with %d errors",
                vectornum, errors);
        end
        $stop;
    end
end
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_TEXTIO.ALL; use STD.TEXTIO.all;

entity testbench3 is -- no inputs or outputs
end;

architecture sim of testbench3 is
    component sillyfunction
        port(a, b, c: in  STD_LOGIC;
             y:      out STD_LOGIC);
    end component;
    signal a, b, c, y:  STD_LOGIC;
    signal y_expected: STD_LOGIC;
    signal clk, reset:  STD_LOGIC;

begin
    -- instantiate device under test
    dut: sillyfunction port map(a, b, c, y);

    -- generate clock
    process begin
        clk <= '1'; wait for 5 ns;
        clk <= '0'; wait for 5 ns;
    end process;

    -- at start of test, pulse reset
    process begin
        reset <= '1'; wait for 27 ns; reset <= '0';
        wait;
    end process;

    -- run tests
    process is
        file tv: text;
        variable L: line;
        variable vector_in: std_logic_vector(2 downto 0);
        variable dummy: character;
        variable vector_out: std_logic;
        variable vectornum: integer := 0;
        variable errors: integer := 0;
    begin
        FILE_OPEN(tv, "example.txt", READ_MODE);
        while not endfile(tv) loop

            -- change vectors on rising edge
            wait until rising_edge(clk);

            -- read the next line of testvectors and split into pieces
            readline(tv, L);
            read(L, vector_in);
            read(L, dummy); -- skip over underscore
```

\$readmemb 将二进制数文件读入 testvectors 数组。\$readmemb 类似，但读取的是十六进制数文件。

下一段代码在时钟上升沿后等待一个时间单位（以避免时钟和数据同时变化时产生混淆），然后根据当前测试向量中的四个比特位设置三个输入和预期输出。

测试平台将生成的输出 y 与预期输出 yexpected 进行比较，如果不匹配则打印错误信息。%b 和 %d 分别表示以二进制和十进制格式打印数值。\$display 是一个系统任务，用于在仿真器窗口中打印信息。例如，\$display ("%b %b", y, yexpected); 会以二进制格式打印 y 和 yexpected 这两个值。%h 则以十六进制格式打印数值。

此过程重复进行，直到测试向量数组中不再有有效的测试向量。\$stop 停止仿真。

请注意，尽管SystemVerilog模块支持多达10,001个测试向量，但在执行完文件中的八个向量后，它将终止仿真。

```
read(L, vector_out);
(a, b, c) <= vector_in(2 downto 0) after 1 ns;
y_expected <= vector_out after 1 ns;

-- check results on falling edge
wait until falling_edge(clk);

if y /= y_expected then
    report "Error: y = " & std_logic'image(y);
    errors := errors + 1;
end if;

vectornum := vectornum + 1;
end loop;

-- summarize results at end of simulation
if (errors = 0) then
    report "NO ERRORS -- " &
        integer'image(vectornum) &
        " tests completed successfully."
        severity failure;
else
    report integer'image(vectornum) &
        " tests completed, errors = " &
        integer'image(errors)
        severity failure;
end if;
end process;
end;
```

VHDL代码使用了超出本章范围的文件读取命令，但它展示了自检测测试平台的大致样貌。

新输入在时钟上升沿施加，输出在时钟下降沿检查。错误发生时立即报告。仿真结束时，测试平台打印施加的测试向量总数以及检测到的错误数量。

HDL示例4.39中的测试平台对于如此简单的电路而言过于复杂。然而，只需修改example.txt文件、实例化新的待测设计（DUT），并更改几行代码来设置输入和检查输出，即可轻松修改以测试更复杂的电路。

4.10 总结

硬件描述语言（HDL）是现代数字设计人员极其重要的工具。一旦你学会了SystemVerilog或VHDL，就能比绘制完整原理图快得多地指定数字系统。调试周期通常也快得多，因为修改只需更改代码，而非繁琐地重新布线原理图。然而，如果你对代码所隐含的硬件没有清晰概念，使用HDL时调试周期可能会变得非常longer。

HDL既用于仿真也用于综合。逻辑仿真是一种在系统转化为硬件之前，在计算机上对其进行测试的强大方法。仿真器允许您检查系统内部信号的数值，而这些信号在物理硬件上可能无法测量。逻辑综合则将HDL代码转换为数字逻辑电路。

编写HDL代码时，最重要的是要记住你是在描述真实的硬件，而不是编写计算机程序。初学者最常见的错误是在没有思考你打算生成的硬件的情况下编写HDL代码。如果你不清楚自己正在描述什么样的硬件，几乎可以肯定无法得到想要的结果。相反，应该先草拟系统的框图，确定哪些部分是组合逻辑、哪些部分是时序电路或有限状态机等。然后，为每个部分编写HDL代码，使用正确的惯用语法来描述你所需的硬件类型。

练习

以下练习可使用您偏好的HDL完成。若有可用的仿真器，请测试您的设计。打印波形图并说明其如何验证设计功能。若有可用的综合工具，请对代码进行综合。打印生成的电路图并解释其为何符合您的预期。

练习4.1 画出以下HDL代码所描述电路的原理图。简化该原理图，使其显示最少数量的门电路。

SystemVerilog	VHDL
<pre>module exercise1(input logic a, b, c, output logic y, z); assign y = a & b & c a & b & ~c a & ~b & c; assign z = a & b ~a & ~b; endmodule</pre>	<pre>VHDL library IEEE; use IEEE.STD_LOGIC_1164.all; entity exercise1 is port(a, b, c: in STD_LOGIC; y, z: out STD_LOGIC); end; architecture synth of exercise1 is begin y <= (a and b and c) or (a and b and not c) or (a and not b and c); z <= (a and b) or (not a and not b); end;</pre>

练习4.2 画出以下HDL代码所描述电路的原理图。简化该原理图，使其显示最少数量的门电路。

SystemVerilog	VHDL
<pre>module exercise2(input logic [3:0] a, output logic [1:0] y); always_comb if (a[0]) y = 2'b11; else if (a[1]) y = 2'b10; else if (a[2]) y = 2'b01; else if (a[3]) y = 2'b00; else y = a[1:0]; endmodule</pre>	<pre>VHDL library IEEE; use IEEE.STD_LOGIC_1164.all; entity exercise2 is port(a: in STD_LOGIC_VECTOR(3 downto 0); y: out STD_LOGIC_VECTOR(1 downto 0)); end; architecture synth of exercise2 is begin process(all) begin if a(0) then y <= "11"; elsif a(1) then y <= "10"; elsif a(2) then y <= "01"; elsif a(3) then y <= "00"; else y <= a(1 downto 0); end if; end process; end;</pre>

练习4.3 编写一个HDL模块，实现四输入异或功能。输入为a_{3:0}，输出为y。

练习4.4 为练习4.3编写一个自检测试平台。创建一个包含全部16个测试用例的测试向量文件。仿真电路并验证其功能正常。

在测试向量文件中引入一个错误，并展示测试平台报告了一个不匹配。

练习4.5 编写一个名为minority的HDL模块。该模块接收三个输入a、b和c，并产生一个输出y。当至少两个输入为FALSE时，y为TRUE。

练习4.6 编写一个十六进制七段数码管显示解码器的HDL模块。该解码器应能处理数字A、B、C、D、E、F以及0-9。

练习4.7 为练习4.6编写一个自检测试平台。创建一个包含全部16个测试用例的测试向量文件。对电路进行仿真并验证其功能正常。在测试向量文件中引入一个错误，并证明测试平台会报告不匹配。

练习4.8 编写一个名为mux8的8:1多路选择器模块，输入为 $s_{2:0}$ 、d0、d1、d2、d3、d4、d5、d6、d7，输出为y。

练习4.9 编写一个结构模块，使用多路选择器逻辑计算逻辑函数 $y = ab + bc + abc = \bar{a} + \bar{b} + \bar{c}$ 。使用练习4.8中的8:1多路选择器。

练习4.10 使用一个4:1多路复用器和所需数量的非门重复练习4.9。

练习4.11 第4.5.4节指出，如果赋值按正确顺序给出，同步器可以用阻塞赋值正确描述。请想出一个无论顺序如何都无法用阻塞赋值正确描述的简单时序电路。

E练习4.12 编写一个八输入优先级电路的HDL模块。

练习4.13 编写一个2:4解码器的HDL模块。

练习4.14 使用三个来自练习4.13的2:4解码器实例和一组三输入与门，编写一个6:64解码器的HDL模块。

练习4.15 编写实现练习2.13中布尔等式的HDL模块。

练习4.16 编写一个HDL模块，实现练习2.26中的电路。

练习4.17 编写一个HDL模块，实现练习2.27中的电路。

练习4.18 编写一个HDL模块，实现练习2.28中的逻辑函数。请特别注意如何处理无关项。

练习4.19 编写一个HDL模块，实现练习2.35中的功能。

练习4.20 编写一个HDL模块，实现练习2.36中的优先编码器。

练习4.21 编写一个HDL模块，实现练习2.37中的改进型优先编码器。

练习4.22 编写一个HDL模块，实现练习2.38中的二进制到温度计码转换器。

练习4.23 编写一个HDL模块，实现问题2.2中的每月天数函数。

练习4.24 画出由以下HDL代码描述的FSM的状态转换图。

SystemVerilog	VHDL
<pre>module fsm2(input logic clk, reset, input logic a, b, output logic y); logic [1:0] state, nextstate; parameter S0 = 2'b00; parameter S1 = 2'b01; parameter S2 = 2'b10; parameter S3 = 2'b11; always_ff @(posedge clk, posedge reset) if (reset) state <= S0; else state <= nextstate; always_comb case (state) S0: if (a ^ b) nextstate = S1; else nextstate = S0; S1: if (a & b) nextstate = S2; else nextstate = S0; S2: if (a b) nextstate = S3; else nextstate = S0; S3: if (a b) nextstate = S3; else nextstate = S0; endcase assign y = (state == S1) (state == S2); endmodule</pre>	<pre>VHDL library IEEE; use IEEE.STD_LOGIC_1164.all; entity fsm2 is port(clk, reset: in STD_LOGIC; a, b: in STD_LOGIC; y: out STD_LOGIC); end; architecture synth of fsm2 is type statetype is (S0, S1, S2, S3); signal state, nextstate: statetype; begin process(clk, reset) begin if reset then state <= S0; elsif rising_edge(clk) then state <= nextstate; end if; end process; process(all) begin case state is when S0 => if (a xor b) then nextstate <= S1; else nextstate <= S0; end if; when S1 => if (a and b) then nextstate <= S2; else nextstate <= S0; end if; when S2 => if (a or b) then nextstate <= S3; else nextstate <= S0; end if; when S3 => if (a or b) then nextstate <= S3; else nextstate <= S0; end if; end case; end process; y <= '1' when ((state = S1) or (state = S2)) else '0'; end;</pre>

练习4.25 画出由以下HDL代码描述的FSM的状态转换图。这种性质的FSM在某些微处理器的分支预测器中使用。

SystemVerilog

```
module fsm1(input  logic clk, reset,
            input  logic taken, back,
            output logic predicttaken);

    logic [4:0] state, nextstate;

    parameter S0 = 5'b00001;
    parameter S1 = 5'b00010;
    parameter S2 = 5'b00100;
    parameter S3 = 5'b01000;
    parameter S4 = 5'b10000;

    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S2;
        else      state <= nextstate;

    always_comb
        case (state)
            S0: if (taken) nextstate = S1;
                else      nextstate = S0;
            S1: if (taken) nextstate = S2;
                else      nextstate = S0;
            S2: if (taken) nextstate = S3;
                else      nextstate = S1;
            S3: if (taken) nextstate = S4;
                else      nextstate = S2;
            S4: if (taken) nextstate = S4;
                else      nextstate = S3;
            default:      nextstate = S2;
        endcase

    assign predicttaken = (state == S4) |
                          (state == S3) |
                          (state == S2 & back);

endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164. all;

entity fsm1 is
    port(clk, reset:  in  STD_LOGIC;
         taken, back: in  STD_LOGIC;
         predicttaken: out STD_LOGIC);
end;

architecture synth of fsm1 is
    type statetype is (S0, S1, S2, S3, S4);
    signal state, nextstate: statetype;
begin
    process(clk, reset) begin
        if reset then state <= S2;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    process(all) begin
        case state is
            when S0 => if taken then
                            nextstate <= S1;
                        else nextstate <= S0;
                        end if;
            when S1 => if taken then
                            nextstate <= S2;
                        else nextstate <= S0;
                        end if;
            when S2 => if taken then
                            nextstate <= S3;
                        else nextstate <= S1;
                        end if;
            when S3 => if taken then
                            nextstate <= S4;
                        else nextstate <= S2;
                        end if;
            when S4 => if taken then
                            nextstate <= S4;
                        else nextstate <= S3;
                        end if;
            when others => nextstate <= S2;
        end case;
    end process;

    -- output logic
    predicttaken <= '1' when
        ((state = S4) or (state = S3) or
         (state = S2 and back = '1'))
    else '0';
end;
```

练习4.26 编写一个SR锁存器的HDL模块。

练习4.27 编写一个用于JK *flip-flop*的HDL模块。该触发器具有输入 clk 、 J 和 K ，以及输出 Q 。在时钟上升沿，如果 $J = K = 0$ ，则 Q 保持其旧值。如果 $J = 1$ ，则将 Q 置为1；如果 $K = 1$ ，则将 Q 复位为0；如果 $J = K = 1$ ，则对 Q 取反。

练习4.28 为图3.18中的锁存器编写一个HDL模块。每个门使用一条赋值语句。为每个门指定1个单位或1纳秒的延迟。仿真该锁存器并证明其正确工作。然后，增加反相器的延迟。延迟需要多长才会导致竞争条件使锁存器功能异常？

练习4.29 编写一个交通灯控制器的HDL模块，来自第3.4.1节。

练习4.30 编写三个HDL模块，用于实现例3.8中的分解式游行模式交通灯控制器。这些模块应分别命名为`controller`、`mode`和`lights`，其输入和输出如图3.33(b)所示。

练习4.31 编写一个HDL模块，描述图3.42中的电路。

练习4.32 编写一个HDL模块，用于实现图3.69（来自练习3.22）中状态转换图所描述的FSM。

练习4.33 编写一个HDL模块，用于实现图3.70（来自练习3.23）中状态转换图所描述的FSM。

练习4.34 编写一个HDL模块，用于改进自练习3.24的交通灯控制器。

练习4.35 为练习3.25中的子蜗牛编写一个HDL模块。

练习4.36 为习题3.26中的汽水机编写一个HDL模块。

练习4.37 编写一个格雷码计数器的HDL模块
练习3.27。

练习4.38 编写一个HDL模块，实现练习3.28中的UP/DOWN格雷码计数器。

练习4.39 为练习3.29中的FSM编写一个HDL模块。

练习4.40 为练习3.30中的FSM编写一个HDL模块。

练习4.41 为问题3.2中的串行二进制补码器编写一个HDL模块。

练习4.42 为练习3中的电路编写一个HDL模块。 31.

练习4.43 为练习3.32中的电路编写一个HDL模块。

练习4.44 为练习3.33中的电路编写一个HDL模块。

练习4.45 为习题3.34中的电路编写一个HDL模块。你可以使用第4.2.5节中的全加器。

SystemVerilog 练习

以下练习特定于SystemVerilog。

练习4.46 在SystemVerilog中，将信号声明为tri意味着什么？

练习4.47 重写HDL示例4.29中的syncbad模块。使用非阻塞赋值，但修改代码以生成具有两个触发器的正确同步器。

练习4.48 考虑以下两个SystemVerilog模块。它们的功能相同吗？画出每个模块所对应的硬件结构。

```

模块 code1(输入 逻辑 clk, a, b, c, 输出 逻辑 y); 逻辑
x;  always_ff @(posedge clk) begin  x <= a & b;  y
<= x | c;  end endmodule  模块2(输入 逻辑 a, b,
c, clk, 输出 逻辑 y); 逻辑 x;  always_ff @(posedge c
lk) begin  y <= x | c;  x <= a & b;  end endmodule

```

练习4.49 若每次赋值中将<=替换为=，重复练习4.48。

练习4.50 以下SystemVerilog模块展示了作者在实验室中看到学生常犯的错误。解释每个模块中的错误并说明如何修正。

```
(a) 模块 latch(input logic    clk, input logic [3:0] d, output reg [3:0] q)
; always @(clk) if (clk) q <= d; endmodule (b) 模块 gates(input logic [
3:0] a, b, output logic [3:0] y1, y2, y3, y4, y5); always @(a) begin y1 =
a & b; y2 = a | b; y3 = a ^ b; y4 = ~(a & b); y5 = ~(a | b); end endmo
odule (c) 模块 mux2(input logic [3:0] d0, d1, input logic    s,
output logic [3:0] y); always @(posedge s) if (s) y <= d1; else y <= d
0; endmodule (d) 模块 twoflops(input logic clk, input logic d0, d1, out
put logic q0, q1); always @(posedge clk) q1 = d1; q0 = d0; endmodul
e (e) 模块 FSM(input logic clk, input logic a, output logic out1, out2);
logic state; // 下一状态逻辑和寄存器 (时序) always_ff @(posedge c
lk) if (state == 0) begin if (a) state <= 1; end else begin if (~a) state
<= 0; end
```

```

        always_comb // 输出逻辑 (组合
    if (state == 0) out1 = 1;        else        out2 = 1; endmo
    dule (f) module priority(input  logic [3:0] a, output logic
    [3:0] y); always_comb if    (a[3]) y = 4'b1000; else if (a
    [2]) y = 4'b0100; else if (a[1]) y = 4'b0010; else if (a[0]) y
    = 4'b0001; endmodule (g) module divideby3FSM(input l
    ogic clk, input  logic reset, output logic out); logic [1:0]
    state, nextstate; parameter S0 = 2'b00; parameter S1 = 2'
    b01; parameter S2 = 2'b10; // 状态寄存器 always_ff @
    (posedge clk, posedge reset) if (reset) state <= S0; else
    state <= nextstate; // 下一状态逻辑 always @(state) case
    (state) S0: nextstate = S1; S1: nextstate = S2; S2: nextst
    ate = S0; endcase // 输出逻辑 assign out = (state == S2)
    ; endmodule (h) module mux2tri(input  logic [3:0] d0, d1
    , input  logic        s, output tri    [3:0] y); tristate t0(d
    0, s, y); tristate t1(d1, s, y); endmodule (i) module floprse
    n(input logic   clk, input logic reset, input logic set,
    input logic [3:0] d, output logic [3:0] q);

```

```

        always_ff @(posedge clk, posedge reset) if (reset) q <=
0; else      q <= d; always @(set) if (set)  q <= 1; end
module

```

```

(j) module and3(input logic a, b, c, output logic y);
    logic tmp; always @(a, b, c) begin tmp
<= a & b; y <= tmp & c; end endmodule

```

VHDL 练习

以下练习专门针对VHDL。

练习4.51 在VHDL中，为什么必须编写

当状态=S0时，q <=赋值为'1'，否则赋值为'0'；而不是简单地q <= (状态= S0)；

练习4.52 以下每个VHDL模块都包含错误。为简洁起见，仅显示架构；假设库使用子句和实体声明正确。解释错误并说明如何修复。

```

(a) 锁存器的架构综合为 begin process(
clk) begin      if clk = '1' then q <= d;
end if;      end process; end; (b) 门电路的
架构过程为 begin process(a) begin
Y1 <= a and b;      y2 <= a or b;      y3
<= a xor b;      y4 <= a nand b;      y5 <=
a nor b;      end process; end;

```

(c) flop的架构综合如下

```
begin process(c
lk)  if rising_edge(clk) then    q <= d;
end;
```

(d) priority的架构综合如下:

```
begin process(al
l) begin if a(3) then y <= "1000"; elsif a(2) then
y <= "0100"; elsif a(1) then y <= "0010"; elsif a
(0) then y <= "0001"; end if; end process; end;
```

(e) 架构综合 divideby3FSM 为 类型 statetype 是 (S
0, S1, S2); 信号 state, nextstate: statetype; 开始 进
程(clk, reset) 开始 如果 reset 则 state <= S0; 否则
如果 rising_edge(clk) 则 state <= nextstate; 结束如
果; 结束进程 进程(state) 开始 情况 state 是 当
0 => nextstate <= S1; 当 1 => nextstate <= S2; 当
2 => nextstate <= S0; 结束情况 结束进程 q <= '1'
当 state = S0 否则 '0'; 结束

(f) architecture struct of mux2 is component tristate port(a:
in STD_LOGIC_VECTOR(3 downto 0); en: in STD_L
OGIC; y: out STD_LOGIC_VECTOR(3 downto 0)); e
nd component; begin t0: tristate port map(d0, s, y); t1: tri
state port map(d1, s, y); end;

(g) floprs的异步架构为

```
begin  process(clk, reset)  b
egin  if reset then      q <= '0';    elsif rising_edge(cl
k) then      q <= d;    end if;  end process;  process(
set)  begin  if set then      q <= '1';    end if;  end
process; end;
```


面试问题

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题4.1 编写一行HDL代码，用一个名为sel的信号门控一个32位总线data，产生一个32位结果。如果sel为真，则结果= data。否则，结果应为全0。

问题4.2 解释SystemVerilog中阻塞赋值和非阻塞赋值的区别。给出示例。

问题4.3 以下SystemVerilog语句的作用是什么？
`assign result = |(data[15:0] & 16'hC820);`

